



FACULTAD DE CIENCIAS EXACTAS, FÍSICAS y NATURALES

Informe de Proyecto Integrador

Un enfoque DevSecOps en el desarrollo IoT para monitorear cultivos

Realizado por

Federico Joaquín Coronati

Matrícula: 40502859

Email: federico.coronati@mi.unc.edu.ar

Celular: 3537659447

Francisco Adrián De la Cruz

Matrícula: 41481173

Email: francisco.delacruz@mi.unc.edu.ar

Celular: 3518135587

Para la obtención del título de Grado en
Ingeniería en Computación

Director

Ing. Miguel Ángel Solinas

Codirector

Ing. Javier Jorge

Resumen

El presente trabajo consiste en añadir prácticas continuas de la cultura DevSecOps al desarrollo de una aplicación con dispositivos Internet of Things para monitorear condiciones aptas para cultivos. Las características más importantes son la Integración Continua, Tests Automatizados y Despliegue Continuo. Además, se hace especial hincapié en la seguridad en todas las etapas del desarrollo y en trabajar con buenas prácticas.

Palabras clave: Internet of Things, Automation, Security

Índice general

1. Marco Teórico	1
1.1. DevOps y Continuous Software Engineering	1
1.2. Seguridad como Requerimiento Funcional	3
1.3. Vulnerabilidades y Amenazas de Seguridad	4
1.3.1. Bases de Datos de Vulnerabilidades	6
1.4. Internet of Things (IoT)	7
1.5. Protocolo MQTT	8
1.6. Control de Versiones con Git	10
1.7. Workflows en GitHub Actions	11
1.8. Runners	12
1.9. Docker	13
1.10. Protocolo HTTP y HTTPS	15
1.11. Proxy	16
2. Requerimientos	18
2.1. Objetivos	18
2.1.1. Objetivo General	18
2.1.2. Objetivos Parciales	18
2.2. Usuarios	18
2.2.1. Clientes	19
2.2.2. Desarrolladores	19
2.3. Características y Requerimientos del Sistema	19
2.3.1. Requerimientos Funcionales	19
2.3.2. Requerimientos No Funcionales	20
2.4. Topología	20
2.5. Análisis de Riesgos	21
2.6. Plan de Implementación	22
2.6.1. Prototipo 1: Envío de datos al Broker MQTT	23
2.6.2. Prototipo 2: Flujo de Trabajo	23
2.6.3. Prototipo 3: Ecosistema de Servicios en Raspberry Pi	24
2.6.4. Prototipo 4: Docker y Testing	24
2.6.5. Prototipo 5: Actualizar por WiFi	25
2.6.6. Prototipo 6: Actualizaciones Automáticas por HTTP	25
2.7. Esquema de Ramas GitFlow	26
3. Selección de los Componentes	28

3.1.	Comparación de Dispositivos IoT	28
3.2.	Selección de Sensores	34
3.3.	Selección del Hardware para el Servidor	36
3.3.1.	Raspberry Pi 4 Model B	36
3.4.	Plataforma de Control de Versiones	38
3.5.	Analizadores de Código	39
3.6.	Compilación y Carga del programa	40
3.6.1.	ESP-IDF	40
3.6.2.	PlatformIO	41
4.	Actividades Desarrolladas	43
4.1.	Preparación Preliminar	43
4.1.1.	Investigación	43
4.1.2.	Configuración del Entorno	43
4.2.	Prototipo 1: Envío de datos al Broker MQTT	44
4.2.1.	La Aplicación	44
4.2.2.	Prototipo Físico	44
4.2.3.	MQTT Broker	45
4.2.4.	Tópicos MQTT	47
4.3.	Prototipo 2: Flujo de Trabajo	49
4.3.1.	Workflow o Flujo de Trabajo	49
4.3.2.	CodeQL	51
4.3.3.	NodeRED	52
4.4.	Prototipo 3: Ecosistema de Servicios en Raspberry Pi	54
4.4.1.	Runner Self-Hosted	54
4.4.2.	Servidor Web Nginx	56
4.4.3.	Servicios de Monitoreo	57
4.4.4.	Nginx Proxy Manager	59
4.4.5.	Portainer	59
4.5.	Prototipo 4: Docker y Testing	60
4.5.1.	Imagen de Docker	60
4.5.2.	Docker Compose	61
4.5.3.	DockerHub	62
4.5.4.	Tests Automatizados	63
4.6.	Prototipo 5: Actualizar por WiFi	63
4.6.1.	Actualizaciones por WiFi	63
4.6.2.	Cppcheck	65
4.6.3.	SonarQube	66
4.7.	Prototipo 6: Actualizaciones Automáticas por HTTP	68
4.7.1.	Actualización Automática por HTTP	68

4.7.2.	Compilación y Disponibilidad del Binario	68
4.7.3.	Certificado HTTPS y Dominio DNS	69
4.7.4.	Gestión de Credenciales con WiFiManager	70
5.	Conclusiones	72
5.1.	Evaluar los Resultados	72
5.1.1.	Verificación de los Requerimientos	72
5.1.2.	Consideraciones Adicionales	73
5.2.	Dificultades Encontradas	74
5.3.	Oportunidades de Mejora	74
5.4.	Agradecimientos	75
6.	Bibliografía	76

Índice de figuras

1.1. Relaciones entre las prácticas continuas CI, CDE y CD.	2
1.2. Etapas del Pipeline DevOps.	3
1.3. Diagrama de Arquitectura Publicación-Suscripción	9
1.4. Diagrama de Estados de Git	10
1.5. Diagrama de Arquitectura de Docker	15
1.6. Diagrama de Proxys	17
2.1. Topología	21
2.2. Esquema de Ramas GitFlow	26
3.1. Placa NodeMCU8266	29
3.2. Placa NodeMCU32S	29
3.3. Placa Raspberry Pi Pico W	29
3.4. Test NOP	30
3.5. Test de pines digitales	31
3.6. Test de operaciones con enteros	32
3.7. Test de operaciones con flotantes	33
3.8. Sensor DHT11	34
3.9. Módulo Sensor de Luz	35
3.10. Sensor de Humedad de Suelo Capacitivo	36
3.11. Raspberry Pi 4 Model B	37
3.12. Proceso de carga del programa en la placa NodeMCU32s	41
3.13. Captura de Pantalla de la extensión PlatformIO para VSCode	42
4.1. 1er prototipo montado en protoboard	45
4.2. Servidor Raspberry Pi en funcionamiento	46
4.3. Captura de Pantalla de la Configuración de MQTT Dash	48
4.4. Captura de Pantalla de las métricas de MQTT Dash	48
4.5. Captura de Pantalla del escaneo con CodeQL	51
4.6. Dashboard de Métricas de los Sensores	53
4.7. NodeRED: Flujo de Nodos	54
4.8. Captura de Pantalla del Runner Self-Hosted	55
4.9. Arquitectura de los Servicios de Monitoreo	58
4.10. Captura de Pantalla del Tablero de Grafana	58
4.11. Flujo de Subdominios	59
4.12. Captura de Pantalla de Portainer	60
4.13. Captura de Pantalla de ElegantOTA	64

4.14. Captura de Pantalla del escaneo con Cppcheck	66
4.15. Captura de Pantalla del las debilidades encontradas por SonarQube . .	67
4.16. Volumen de Docker para el Firmware	69

Índice de tablas

2.1. Tabla de Análisis de Requerimientos	22
3.1. Test NOP	30
3.2. Test de pines digitales	30
3.3. Test de operaciones con enteros	31
3.4. Test de operaciones con flotantes	32
3.5. Otras características	33
3.6. Comparativa de plataformas de alojamiento	38

Índice de extractos de código

1.1. Codigo Vulnerable	5
4.1. Codigo Workflow Simple	50

1. Marco Teórico

1.1. DevOps y Continuous Software Engineering

En el entorno actual de desarrollo de software, donde la velocidad, la calidad y la confiabilidad son fundamentales, ha surgido una nueva filosofía llamada **DevOps**. Consiste en unir las prácticas de desarrollo de software (Dev) con las operaciones de infraestructura (Ops), con el objetivo de acelerar la entrega de aplicaciones y mejorar la colaboración entre los equipos de desarrollo y operaciones [6]. Este enfoque se ha vuelto cada vez más popular debido a los beneficios que ofrece en términos de eficiencia y agilidad en el ciclo de vida del software [32]. DevOps promueve la integración y la automatización en todas las etapas del ciclo de vida de desarrollo de software. A pesar de ser una tendencia en la industria, DevOps carece de una definición ampliamente aceptada [35]. Definir DevOps es difícil debido a la considerable superposición con las **prácticas continuas** [52]. Por esta razón, Stahl et al. [52] presentan definiciones para diferenciar DevOps y prácticas continuas entre sí.

DevOps fomenta una cultura de **colaboración**, donde los equipos de desarrollo y operaciones trabajan juntos desde el principio. La comunicación fluida, la confianza mutua y el enfoque en objetivos comunes son fundamentales para el éxito de DevOps [32].

La **automatización** desempeña un papel crucial en DevOps. Permite la implementación rápida y repetible de infraestructuras y aplicaciones, así como pruebas y despliegues continuos. La automatización reduce errores, mejora la eficiencia y acelera el tiempo de comercialización [18].

DevOps se enfoca en el **monitoreo constante** de aplicaciones y sistemas en producción. El monitoreo proporciona información valiosa sobre el rendimiento, la disponibilidad y la experiencia del usuario, lo que permite realizar mejoras continuas y rápidas [6].

En la última década, el desarrollo de software ha evolucionado en la dirección del **Continuous Software Engineering (CSE)**, que tiene como objetivo establecer un movimiento continuo en las actividades de la ingeniería de software, en lugar de un conjunto de actividades discretas realizadas por diferentes equipos o departamentos [21]. DevOps se basa en las **prácticas continuas**, lo que significa que los equipos pueden entregar cambios y nuevas características de manera rápida y frecuente. Esto se logra mediante la automatización de pruebas, la integración continua y la implementación

continua [53].

En la industria, las **prácticas continuas** relacionadas con el desarrollo cada vez se utilizan con mayor frecuencia y ya están bien establecidas [7]. Gran parte de la investigación sobre prácticas continuas se ha realizado sobre prácticas relacionadas con el desarrollo [48], [49], [47], [58]. Según estos estudios, **Continuous Integration (CI)**, **Continuous Delivery (CDE)** y **Continuous Deployment (CD)** pueden verse como las prácticas continuas más populares en el dominio de CSE [52].

Continuous Integration (CI) es la práctica de desarrollo de software donde los miembros integran con frecuencia su trabajo (p. ej., cambios de código) en la rama principal, normalmente a diario [52]. Estos cambios son validados por compilaciones y pruebas automatizadas [36]. Al hacerlo, los desarrolladores pueden detectar y abordar fallas de integración de manera temprana y lo más rápido posible [21].

Continuous Delivery (CDE) tiene como objetivo mantener el software en un estado de implementación confiable o listo para producción en todo momento [10]. Para lograr este objetivo, el software debe pasar pruebas y controles de calidad pertinentes en un entorno de prueba (p. ej., pruebas de aceptación). Sin embargo, la implementación en el entorno de producción se realiza de forma manual, donde un miembro del equipo con la autoridad pertinente decide cuándo y qué resultados listos para la producción se deben entregar al cliente (es decir, enfoque basado en extracción) [49].

Continuous Deployment (CD) tiene como objetivo ampliar CDE mediante la implementación automática y continua de despliegues de software en un entorno de producción. Se supone que se han superado todos los objetivos de calidad requeridos [56], [48]. Los cambios de software se implementan directamente sobre producción a través del “pipeline” de implementación sin intervención humana [47].

En la Figura 1.1 se muestra la relación entre estas prácticas continuas, que se va a considerar en este Proyecto Integrador.

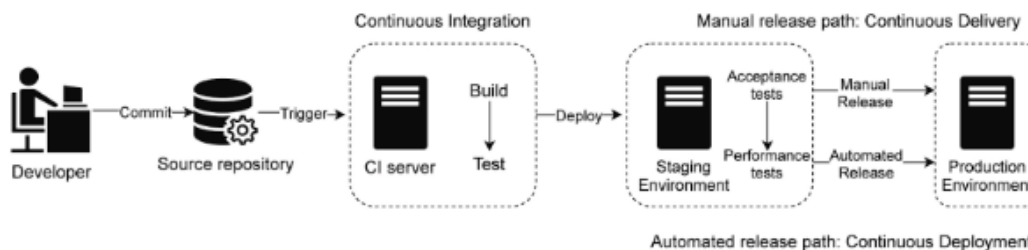


Figura 1.1: Relaciones entre las prácticas continuas CI, CDE y CD.

Se emplea el término **Pipeline** para referirse al flujo de trabajo, es decir, las tareas a realizar en todas las etapas del desarrollo de software. En la Figura 1.2 se ilustran las etapas que componen un pipeline típico de la filosofía DevOps.

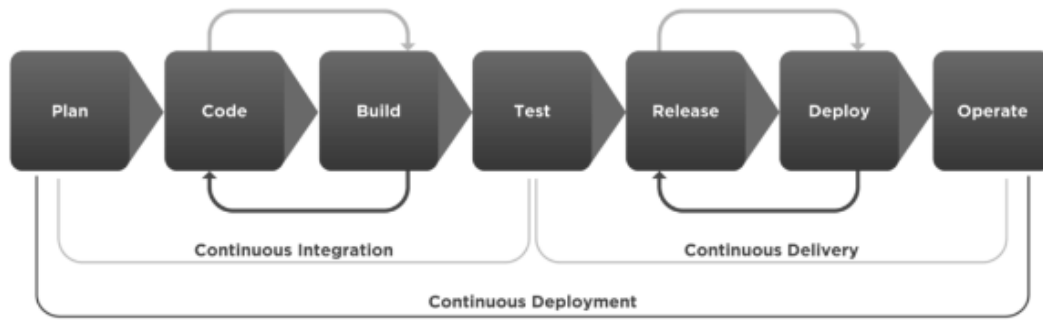


Figura 1.2: Etapas del Pipeline DevOps.

Esta nueva cultura de desarrollo trae consigo importantes beneficios, siendo los principales:

- **Reducción de Tiempos:** La automatización reduce las tareas que deben realizar los operarios y por lo tanto se reducen los tiempos.
- **Menos Errores:** Al definir flujos de trabajo automáticos repetibles se evita depender del conocimiento y experiencia del operario, lo que resulta en menos errores.
- **Escalabilidad y Confiabilidad:** Las prácticas como la infraestructura como código y la escalabilidad automática permiten optimizar los recursos que se usan y escalar según la demanda, lo que aumenta la confiabilidad y permite adaptarse a las demandas cambiantes.

A pesar de los beneficios, la implementación de DevOps puede presentar desafíos. Algunos desafíos comunes incluyen la resistencia al cambio, la complejidad de la automatización y la necesidad de una cultura organizativa receptiva [32].

Como concepto adicional, se define **DevSecOps** (desarrollo, seguridad, y operaciones, en inglés) como la automatización y la integración de la seguridad en todas las etapas del ciclo de vida del software, desde el diseño inicial hasta la entrega de software, pasando por la integración, la prueba y la implementación.

1.2. Seguridad como Requerimiento Funcional

En la actualidad, la **seguridad** se ha convertido en un aspecto fundamental en el desarrollo de software. Los avances tecnológicos y la creciente interconexión de sistemas han aumentado la exposición a **vulnerabilidades** y **amenazas**. Por lo tanto, es crucial considerar la seguridad como un requerimiento funcional en el ciclo de vida del software, es decir, que un sistema o software debe diseñarse para protegerse y

mantener la confidencialidad, integridad y disponibilidad de los datos y recursos. Esto implica identificar y evaluar los riesgos potenciales, así como establecer medidas de protección adecuadas.

Es responsabilidad de las organizaciones y desarrolladores considerar la seguridad desde las etapas iniciales del proceso de desarrollo y mantenerla como una preocupación constante a lo largo del ciclo de vida del software [19]. Su consideración y aplicación adecuadas garantizan la confianza de los usuarios, protegen los datos sensibles y previenen ataques maliciosos, además de resultar en una disminución de costos asociados a solucionar imprevistos de seguridad o incluso llegar a descartar el trabajo de varios meses por no haber tomado en cuenta los riesgos de seguridad.

Si un sistema es vulnerable a ataques o fallas de seguridad, los usuarios pueden perder la confianza en la organización y en los productos o servicios ofrecidos. Por lo tanto, la implementación de medidas de seguridad robustas es indispensable para garantizar la satisfacción del usuario y preservar la reputación de la organización.

En un entorno donde la recopilación y el procesamiento de datos son cada vez más comunes, la protección de datos sensibles se ha convertido en una preocupación clave. Los sistemas deben garantizar que la información confidencial de los usuarios, como datos personales y financieros, esté protegida contra accesos no autorizados. La seguridad como requerimiento funcional implica implementar mecanismos de cifrado, autenticación y control de acceso adecuados para salvaguardar estos datos.

1.3. Vulnerabilidades y Amenazas de Seguridad

En el entorno digital actual, las organizaciones se enfrentan a un panorama de vulnerabilidades y amenazas de seguridad cada vez más complejo.

Una **vulnerabilidad** es una debilidad existente en un sistema que puede ser utilizada por una persona malintencionada para comprometer su seguridad. Una **amenaza** es la acción que se ejerce a través de la explotación de una vulnerabilidad o el engaño que se efectúa a una persona para acceder a un sistema no autorizado. Las causas más comunes que dejan un sistema expuesto a amenazas son la falta de actualizaciones de seguridad, la ingeniería social y el malware, en ocasiones agravándose por desconocer las fallas de seguridad que tiene el sistema.

Los sistemas operativos, aplicaciones y dispositivos deben ser **actualizados** regularmente para remediar las vulnerabilidades conocidas que se van encontrando por la comunidad. Sin embargo, muchos usuarios y organizaciones no aplican actualizaciones de forma oportuna, lo que deja sus sistemas expuestos.

Los ataques de **ingeniería social** implican el engaño de los usuarios para obtener información confidencial, como contraseñas o datos de acceso. Los atacantes utilizan técnicas de manipulación psicológica y engaño para persuadir a los usuarios a revelar información sensible. La falta de capacitación de los usuarios en materia de seguridad puede exponer a las organizaciones a este tipo de ataques.

El **malware** es software malicioso diseñado para dañar, acceder de manera no autorizada o controlar sistemas informáticos. Los tipos de malware más comunes incluyen virus, gusanos, troyanos y ransomware. Los atacantes pueden distribuir malware a través de sitios web comprometidos, archivos adjuntos de correo electrónico o dispositivos USB infectados.

Por otro lado, los desarrolladores pueden introducir vulnerabilidades en las aplicaciones sin notarlo, simplemente escribiendo código que hace uso de funciones vulnerables o que emplea malas prácticas. Es importante incluir herramientas que permitan detectar de manera temprana cuando se añaden vulnerabilidades al código fuente de las aplicaciones para trabajar en solucionar los problemas y no dar lugar a amenazas de seguridad.

Un ejemplo de vulnerabilidad introducida es la **comparación de tipos numéricos de distinto ancho**, si se tiene un bucle while cuya condición implica una comparación entre dos valores numéricos, y estos valores son de distinto ancho de palabra, se podría dar el caso de un bucle infinito debido al **overflow** del valor menos ancho.

A continuación se presenta un fragmento de código para ejemplificar esta vulnerabilidad

```
1  uint8_t A;  
2  uint16_t B = UINT8_MAX + 1;  
3  while(A < B)  
4  {  
5      A++;  
6  }
```

Extracto de código 1.1: Código Vulnerable

Obsérvese que el valor de **A** sólo puede estar comprendido entre 0 a 255, debido a su ancho de 8 bits, y si **B** toma un valor mayor a 255, en este caso **UINT8_MAX + 1**, que equivale a 256, la condición del bucle **while** será siempre verdadera y el programa quedará ejecutándose infinitamente, debido a que la variable **A** nunca va a llegar al valor 256 porque su ancho de 8 bits provoca un desbordamiento (overflow) que reinicia a 0 el valor de **A** y se vuelve a repetir el proceso.

Este es sólo un ejemplo y como este hay muchos otros tipos de vulnerabilidades

que se pueden introducir al codificar sin siquiera notarlo, es por ello que en el presente proyecto se prestará especial atención a añadir herramientas de análisis de código para la detección temprana de vulnerabilidades antes de que la versión de la aplicación en desarrollo salga efectivamente a producción, donde podría ser explotada por los potenciales atacantes.

1.3.1. Bases de Datos de Vulnerabilidades

Es importante mencionar que existe una base de datos que agrupa información acerca de las vulnerabilidades conocidas hasta la fecha, llamada **Common Vulnerabilities and Exposures (CVE)**, en la que cada referencia tiene un número de identificación **CVE-ID**, descripción de la vulnerabilidad, qué versiones del software están afectadas, su posible solución al fallo (si es que existe) y referencias a publicaciones donde se ha dado a conocer la vulnerabilidad o se demuestra su explotación. Es mantenida por **The MITRE Corporation**, una organización estadounidense sin fines de lucro que realiza investigación y desarrollo de tecnologías de la información y aporta estos datos al gobierno de Estados Unidos.

Además, la organización MITRE también mantiene el **Common Weakness Enumeration (CWE)**, el cual es un sistema de categorías para las vulnerabilidades del software. Cada una se identifica con su **CWE-ID**, del mismo modo que con CVE. Se sustenta en un proyecto comunitario con el objetivo de comprender las fallas en el software y crear herramientas automatizadas que se puedan utilizar para identificar, corregir y prevenir esas fallas. CWE tiene más de 600 categorías, incluidas clases para buffer overflows, path/directory tree traversal errors, condiciones de carrera, cross-site scripting, hard-coded passwords y generación insegura de números aleatorios.

En resumen, **CWE** es un estándar para clasificar debilidades en general y no está relacionada con ningún sistema en específico, mientras que **CVE** es un estándar para listar vulnerabilidades específicas de un producto o sistema. Estas dos bases de datos continúan creciendo día tras día cada vez que se descubren nuevas vulnerabilidades y debilidades generales.

Es habitual que las herramientas de análisis de código brinden información sobre las vulnerabilidades encontradas, una breve descripción junto con sugerencias para solucionar el problema y un enlace o referencia al CWE, ya que es más probable que una vulnerabilidad introducida al codificar se encuentre dentro de las debilidades generales en CWE y no en CVE.

1.4. Internet of Things (IoT)

El **Internet de las cosas** (IoT, por sus siglas en inglés) es una red de dispositivos físicos, vehículos, electrodomésticos y otros objetos que están integrados con sensores, software y conectividad para permitir la recopilación y el intercambio de datos. Estos dispositivos están conectados a través de Internet, lo que les permite comunicarse entre sí y realizar acciones inteligentes basadas en la información recopilada [4].

IoT ha abierto un mundo de posibilidades y ha generado beneficios significativos en diversas áreas. En primer lugar, la automatización y la eficiencia son dos beneficios clave. Los dispositivos IoT pueden recopilar datos en tiempo real, permitiendo el monitoreo y control de los procesos. Esto conduce a una mayor eficiencia en la producción industrial, la gestión energética y el mantenimiento de infraestructuras [29].

Además, IoT ha mejorado nuestra calidad de vida en el ámbito doméstico. Los hogares inteligentes, equipados con dispositivos IoT, permiten a los usuarios controlar y gestionar remotamente electrodomésticos, sistemas de seguridad y energía, lo que brinda comodidad y seguridad. Asimismo, el IoT ha impulsado el desarrollo de ciudades inteligentes, donde sensores y dispositivos conectados mejoran la gestión del tráfico, la recolección de residuos, el consumo de energía y la seguridad pública. Al campo de la automatización de tareas del hogar con dispositivos inteligentes se le llama **domótica**, y está relacionado con IoT porque muchas veces estos dispositivos necesitan conectarse entre sí.

Sin embargo, IoT también plantea desafíos y preocupaciones significativas. Uno de los principales desafíos es la seguridad. Dado que los dispositivos IoT están conectados a Internet, son vulnerables a ataques cibernéticos. La falta de estándares de seguridad y la proliferación de dispositivos no seguros pueden dar lugar a brechas de seguridad y comprometer la privacidad de los usuarios. La recopilación masiva de datos genera preocupaciones sobre la privacidad y la ética. Con la cantidad de dispositivos IoT en aumento, se recopilan y analizan enormes cantidades de datos personales. Es fundamental establecer regulaciones y prácticas claras para garantizar la protección de la privacidad y el uso ético de estos datos [5].

En el presente proyecto, se presentarán prácticas y herramientas para trabajar sobre un proyecto IoT sencillo, y se hará especial énfasis en las buenas prácticas de desarrollo y en la seguridad.

1.5. Protocolo MQTT

El protocolo **MQTT (Message Queuing Telemetry Transport)** es un protocolo de mensajería ligero diseñado para facilitar la comunicación eficiente y confiable en el contexto del Internet de las Cosas (IoT). Desarrollado por IBM en la década de 1990, MQTT se ha convertido en uno de los estándares más utilizados para la conectividad de dispositivos en entornos distribuidos. Se basa en la arquitectura cliente-servidor, donde los dispositivos se dividen en dos categorías principales: los **clientes MQTT** y los **brokers MQTT**. Los clientes MQTT pueden ser tanto productores como consumidores de información, y se conectan a un broker MQTT para intercambiar mensajes [31].

Los mensajes en MQTT se organizan en **tópicos**, que son identificadores alfanuméricos que representan los diferentes canales de comunicación. Los clientes MQTT pueden suscribirse a uno o varios tópicos y publicar mensajes en ellos. Cuando un cliente envía un mensaje a un tópico, el broker es el encargado de reenviar el mensaje al tópico para que los suscriptores lo reciban y puedan procesarlo.

El protocolo MQTT sigue un enfoque de **Publicación-Suscripción**, lo que significa que los clientes no necesitan establecer conexiones directas entre sí. En cambio, todos los mensajes pasan a través del broker, que actúa como intermediario para la distribución eficiente de los mensajes. Se dice que hay acoplamiento temporal cuando tanto el emisor y el receptor coinciden en el tiempo para comunicarse. En el caso del protocolo MQTT, la coexistencia en el tiempo entre los clientes no es una condición necesaria para establecer el intercambio de datos. Esto reduce la carga de red y simplifica la administración de la conectividad en entornos IoT masivos.

La figura 1.3 muestra un diagrama de la arquitectura Publicación-Suscripción, donde se puede apreciar al servidor o Broker MQTT y los clientes que publican y reciben mensajes de los tópicos a los cuales se suscriben.

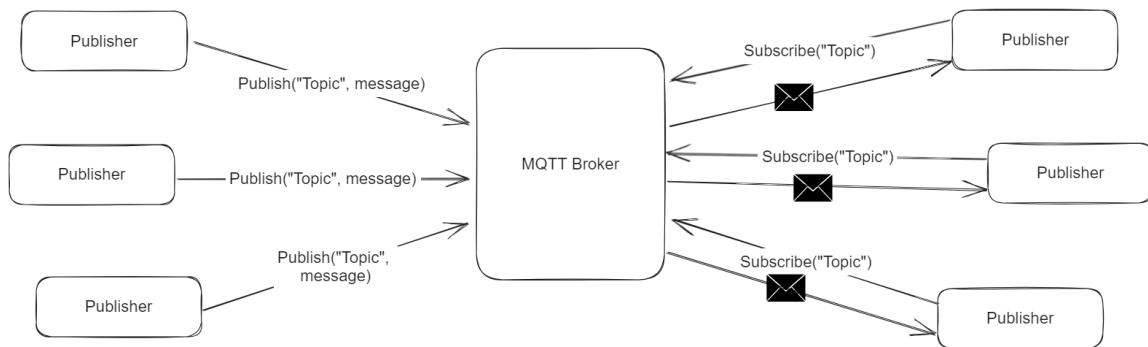


Figura 1.3: Diagrama de Arquitectura Publicación-Suscripción

MQTT ofrece varias características clave que resultan convenientes para aplicaciones IoT:

- **Ligero y eficiente:** MQTT fue diseñado teniendo en cuenta la eficiencia en redes con ancho de banda limitado y dispositivos con recursos limitados, como sensores o dispositivos embebidos. El protocolo es ligero en términos de consumo de energía, ancho de banda y uso de memoria, lo que lo hace ideal para entornos IoT.
- **Calidad de servicio adaptable:** MQTT ofrece tres niveles de calidad de servicio (QoS, del inglés Quality of Service) para garantizar la entrega confiable de mensajes en entornos con conectividad inestable. Los niveles de QoS varían desde "como máximo una vez" (QoS 0) hasta "al menos una vez" (QoS 1) y "exactamente una vez" (QoS 2), proporcionando opciones flexibles según los requisitos de la aplicación.
- **Conectividad persistente:** MQTT permite conexiones persistentes entre los clientes y el broker MQTT. Esto significa que los clientes pueden desconectarse temporalmente de la red y luego reconectarse sin perder mensajes. Además, MQTT admite mecanismos de sesión para mantener el estado y la configuración del cliente, lo que facilita la gestión de dispositivos IoT.
- **Seguridad:** MQTT proporciona opciones de seguridad para proteger la comunicación entre los clientes y el broker. Puede implementarse a través de canales seguros utilizando TLS/SSL, y también admite mecanismos de autenticación y autorización para garantizar que sólo los clientes autorizados puedan acceder a los mensajes y a los tópicos.

El protocolo MQTT se utiliza ampliamente en una variedad de aplicaciones IoT,

como la ya mencionada domótica, el monitoreo y control de procesos productivos, y la agricultura de precisión, siendo esta última cada vez más popular ya que permite a los agricultores tomar decisiones basadas en los datos observados para maximizar los rendimientos y optimizar el uso de recursos.

1.6. Control de Versiones con Git

El **control de versiones** es un componente fundamental en el desarrollo de software, ya que permite gestionar de manera eficiente los cambios realizados en el código fuente a lo largo del tiempo. **Git** es uno de los sistemas de control de versiones más populares y ampliamente utilizados en la actualidad, conocido por su velocidad, flexibilidad y distribución [9].

Git es un sistema de control de versiones distribuido, lo que significa que cada desarrollador tiene una copia completa del repositorio en su máquina local. Esto permite trabajar de forma independiente, realizar cambios y explorar la historia del proyecto sin conexión a la red. Los tres estados principales en Git son: directorio de trabajo, área de preparación (staging) y repositorio.

En la figura 1.4 se puede observar un diagrama de estados donde se tienen las 3 áreas de trabajo por la izquierda. Opcionalmente se puede alojar el repositorio en una plataforma en línea, que se representa en el sector derecho del diagrama.

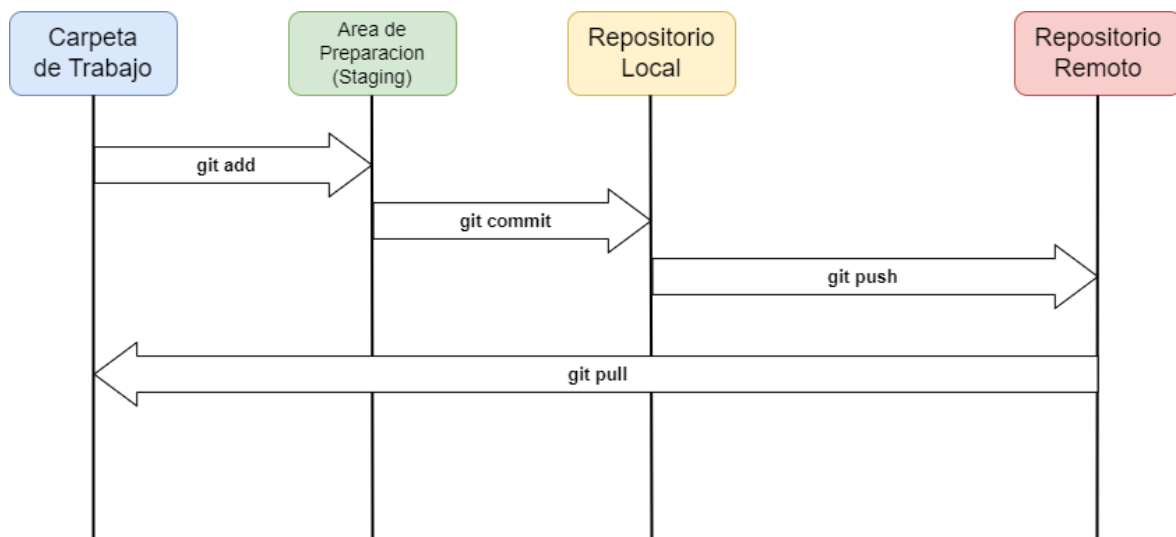


Figura 1.4: Diagrama de Estados de Git

1.7. Workflows en GitHub Actions

Los **Workflows** en GitHub Actions son una poderosa herramienta de automatización que permite a los desarrolladores definir, personalizar y ejecutar flujos de trabajo en sus proyectos. Estos flujos de trabajo están compuestos por una serie de tareas individuales, llamadas **acciones**, que se ejecutan en respuesta a eventos específicos dentro de un repositorio de **GitHub**.

Un **Workflow** en GitHub Actions es una secuencia automatizada de tareas que se ejecutan en respuesta a eventos desencadenados por cambios en un repositorio. Los eventos pueden incluir confirmaciones (commits), aperturas de solicitudes de extracción (pull requests), creación de etiquetas (tags) y otras acciones que ocurren en el repositorio. Cada workflow se define en un archivo YAML ubicado en la carpeta `.github/workflows` de un repositorio de GitHub.

Un archivo de workflow especifica el conjunto de **acciones** que se deben ejecutar, el orden en que deben ocurrir y las condiciones bajo las cuales deben ejecutarse. Las acciones son comandos individuales o scripts que pueden ser proporcionados por la comunidad o creados por el propio desarrollador. GitHub Actions ofrece un amplio catálogo de acciones predefinidas y también permite la creación de acciones personalizadas para adaptarse a las necesidades específicas del proyecto [25].

Existen varias ventajas en el uso de workflows en GitHub Actions para proyectos de desarrollo:

- **Automatización flexible:** Los workflows permiten automatizar tareas repetitivas, como pruebas de integración, compilación de código, despliegue a entornos de producción, entre otros. Esto ahorra tiempo y reduce los errores humanos.
- **Integración Continua y Entrega Continua (CI/CD):** GitHub Actions facilita la implementación de prácticas de CI/CD al proporcionar una forma sencilla de definir flujos de trabajo para la construcción, pruebas y despliegue automáticos de aplicaciones.
- **Amplia compatibilidad:** GitHub Actions es compatible con una amplia variedad de lenguajes de programación, plataformas y servicios en la nube, lo que permite una integración sin problemas con diferentes tecnologías utilizadas en un proyecto.
- **Comunidad:** La comunidad de GitHub ha creado numerosas acciones que abordan diferentes casos de uso. Además, GitHub Actions permite la creación de acciones personalizadas, lo que brinda flexibilidad para adaptar los flujos de trabajo a las necesidades específicas del proyecto.

GitHub Actions ofrece un nivel gratuito que permite ejecutar workflows en repositorios públicos y privados. Con el nivel gratuito, los usuarios tienen acceso a 2,000 minutos de tiempo de ejecución por mes, lo que es suficiente para proyectos pequeños y medianos. Si se supera este límite, es posible actualizar a una cuenta de pago para obtener más minutos de tiempo de ejecución y características adicionales [24].

Aunque el presente Proyecto Integrador es un prototipo y puede funcionar con el nivel gratuito de GitHub Actions, se busca presentar una solución escalable y personalizable, sin llegar al plan de pago sino que empleando un servidor local conocido como Runner, y se indica en la siguiente sección.

1.8. Runners

Un **Runner**, en el contexto de GitHub, es un agente de ejecución que puede correr tareas o acciones dentro de un entorno específico. Existen dos tipos principales de runners: los runners alojados en la nube de GitHub y los runners self-hosted, es decir, que corren en un entorno local.

En el caso de los **runners self-hosted**, los usuarios tienen la opción de alojarlos en sus propios servidores, máquinas virtuales o incluso en entornos de contenedores locales. Esto brinda un mayor control y flexibilidad sobre los recursos utilizados en la ejecución de los flujos de trabajo, así como la capacidad de adaptarse a los requerimientos específicos de los proyectos.

Para utilizar un runner en modo self-hosted, se requiere contar con una máquina o servidor en la que se pueda alojar. Puede ser una máquina física, una máquina virtual o un contenedor. Esta máquina debe tener conexión a internet para comunicarse con GitHub y descargar los flujos de trabajo y las acciones necesarias.

Existen algunas diferencias clave entre los runners self-hosted y los runners alojados en la nube de GitHub:

- **Control y seguridad:** Con los runners self-hosted, los usuarios tienen un mayor control sobre el entorno de ejecución, lo que les permite personalizar la configuración y mantener un mayor nivel de seguridad, especialmente cuando se manejan datos sensibles.
- **Escalabilidad y recursos:** Los runners en la nube de GitHub son altamente escalables y se adaptan automáticamente a las necesidades de carga de trabajo. Los runners self-hosted tienen limitaciones en cuanto a los recursos disponibles, ya que dependen de la capacidad del sistema en el que se alojan.

- **Costos:** Para proyectos pequeños es suficiente con los runners en la nube de GitHub, sin embargo para proyectos medianos y grandes es más rentable su alternativa local, ya que no hay costos adicionales asociados a la infraestructura de ejecución proporcionada por GitHub, siempre y cuando se cuente con el personal para darle un mantenimiento adecuado.

Los runners self-hosted son especialmente útiles en proyectos que requieren entornos personalizados, es decir, configuraciones específicas de hardware y software [26]. Además, si los requisitos de seguridad son estrictos, los runners self-hosted permiten tener mayor control sobre la aplicación y el acceso a los datos. En este Proyecto Integrador, será especialmente útil contar con un runner self-hosted porque permite la comunicación con otros servicios que se detallarán más adelante en el presente informe.

1.9. Docker

Docker es una plataforma de virtualización a nivel de sistema operativo que permite a los desarrolladores empaquetar aplicaciones y todas sus dependencias en **contenedores** livianos y autónomos. A diferencia de la virtualización tradicional basada en hipervisores, Docker aprovecha las funcionalidades de aislamiento del kernel del sistema operativo subyacente para garantizar la consistencia en la ejecución de aplicaciones en diferentes entornos. Estos contenedores brindan una unidad coherente de implementación, lo que facilita el desarrollo colaborativo y la escalabilidad de las aplicaciones [45].

Una de las principales ventajas de Docker radica en su capacidad para abordar el desafío que implica instalar aplicaciones en servidores, ya que la falta de dependencias implica que funciona en algunas máquinas y no en otras. Gracias a la tecnología de los contenedores se garantiza que las aplicaciones se ejecuten de manera coherente en cualquier entorno. Al empaquetar todas las dependencias, los desarrolladores pueden eliminar las disparidades entre los entornos de desarrollo, pruebas y producción. Esto reduce significativamente los conflictos y los tiempos de inactividad causados por las diferencias de configuración entre los sistemas.

Otra ventaja clave de Docker es su capacidad para facilitar la implementación y escalabilidad de aplicaciones. Al dividir las aplicaciones en componentes más pequeños y escalables, los contenedores permiten una distribución eficiente de la carga de trabajo en múltiples servidores. Además, Docker habilita la **orquestación de contenedores** a través de herramientas como **Kubernetes**, lo que simplifica la administración de clústeres de contenedores y brinda mayor disponibilidad y confiabilidad a las aplicaciones [30].

Una de las prácticas más importantes del presente Proyecto Integrador implica colocar los servicios y aplicaciones en contenedores de Docker para facilitar el despliegue a los desarrolladores IoT, esto se detalla más adelante.

Es importante distinguir el concepto de contenedor del de imagen. Los **contenedores** de Docker son instancias en ejecución de una aplicación, empaquetada en un pequeño entorno que contiene sus dependencias instaladas y archivos de configuración listos para ser usados. A menudo una aplicación utiliza uno o varios contenedores conectados entre sí. Por otro lado, una **imagen** es un archivo que contiene la información para crear un contenedor, es decir contiene un pequeño sistema operativo, el código de la aplicación, los archivos de configuración y otras dependencias. Las imágenes se usan para ejecutar contenedores, por lo que podemos instanciar varios contenedores a partir de la misma imagen, en ocasiones con ciertos parámetros y en otras de manera automática con un archivo conocido como **Docker Compose**, que indica cuántos contenedores instanciar y a partir de cuáles imágenes, además de la posibilidad de interconectarlos entre sí.

Además, existe la posibilidad de subir estas imágenes a la nube, a plataformas conocidas como **registry**, por ejemplo **DockerHub**, que es el alojamiento web que ofrece Docker de manera gratuita para repositorios públicos, y del cual se hará uso en el presente proyecto brindando los detalles más adelante. DockerHub permite pushear cambios en una imagen y que las personas que están usándola puedan descargar la última versión en sus sistemas.

En la figura 1.5 se puede apreciar un diagrama de arquitectura de Docker, donde el sistema anfitrión o host instancia varios contenedores a partir de imágenes obtenidas de la plataforma online o Registry.

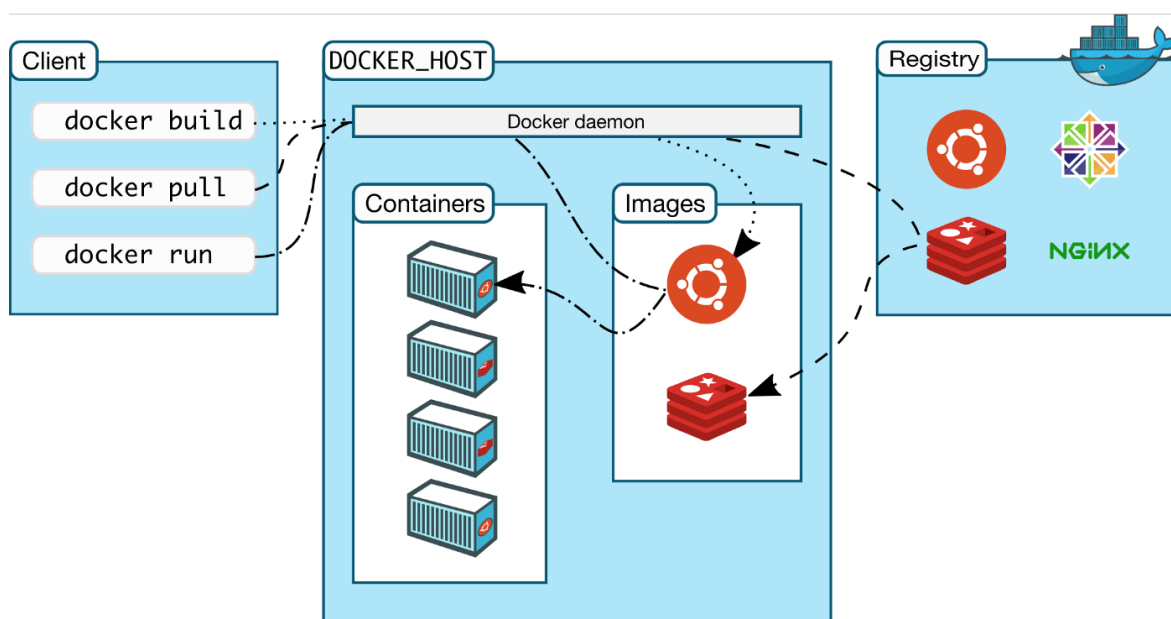


Figura 1.5: Diagrama de Arquitectura de Docker

1.10. Protocolo HTTP y HTTPS

El **Protocolo de Transferencia de Hipertexto (HTTP)**, por sus siglas en inglés) es un protocolo de aplicación utilizado para la comunicación en Internet. HTTP se basa en el modelo cliente-servidor, donde el cliente realiza solicitudes al servidor y este último responde con los datos solicitados.

HTTP se basa en el estilo arquitectónico REST (Representational State Transfer), que promueve la interoperabilidad, escalabilidad y simplicidad en los sistemas distribuidos. REST utiliza un conjunto de operaciones bien definidas, conocidas como métodos de solicitud HTTP, para manipular los recursos web [20]. Los métodos de solicitud más comunes son **GET**, **POST**, **PUT** y **DELETE**, que permiten recuperar, crear, actualizar y eliminar recursos, respectivamente.

HTTP es un protocolo sin estado, lo que significa que cada solicitud y respuesta son independientes y no tienen memoria de las interacciones previas. Esto permite que las solicitudes se realicen de forma aislada y no afecten el estado del servidor. Sin embargo, para manejar estados y sesiones en aplicaciones web, se utilizan mecanismos como cookies y tokens de autenticación [3].

Otra característica importante de HTTP es que utiliza el modelo petición-respuesta. El cliente envía una solicitud HTTP al servidor, que procesa la solicitud y envía una respuesta con un código de estado que indica si la solicitud se completó con éxito o si

ocurrió algún error.

Por otro lado, el **Protocolo Seguro de Transferencia de Hipertexto (HTTPS)** es una extensión del protocolo HTTP que proporciona una capa adicional de seguridad en las comunicaciones web. HTTPS utiliza el cifrado y la autenticación para proteger la confidencialidad e integridad de los datos transmitidos entre el cliente y el servidor. Esta capa de seguridad se basa en el uso de los protocolos **Secure Sockets Layer (SSL)** o **Transport Layer Security (TLS)**.

HTTPS utiliza certificados digitales para autenticar los servidores web y establecer comunicaciones cifradas. Estos certificados son emitidos por autoridades de certificación confiables y garantizan que el servidor al que el cliente se conecta es legítimo y no ha sido alterado. La autenticación del servidor es esencial para prevenir ataques de suplantación y garantizar la confianza del cliente en el sitio web.

Además, HTTPS utiliza técnicas de cifrado para proteger la confidencialidad de los datos transmitidos. HTTPS utiliza algoritmos de cifrado asimétrico y simétrico para asegurar la privacidad de la comunicación. El cifrado asimétrico se utiliza para establecer una clave de sesión compartida entre el cliente y el servidor, mientras que el cifrado simétrico se utiliza para cifrar y descifrar los datos durante la transferencia.

La adopción de HTTPS ha aumentado significativamente debido a la creciente preocupación por la seguridad en la web. El porcentaje de páginas web cargadas a través de HTTPS ha alcanzado más del 90 % en algunas regiones.

1.11. Proxy

Un **Proxy** es un servidor que actúa como intermediario entre los dispositivos de una red y el resto de Internet. Esencialmente, un proxy recibe las solicitudes de los clientes y las envía en su nombre a los servidores de destino. A continuación, devuelve las respuestas del servidor al cliente original. Esto permite que los usuarios accedan a los recursos de Internet sin que su dirección IP real sea revelada, ya que el proxy oculta la identidad y ubicación del cliente.

El funcionamiento de un proxy se basa en la interceptación y reenvío de las solicitudes. Cuando un usuario envía una solicitud para acceder a un recurso en Internet, esta solicitud es enviada primero al proxy. El proxy, a su vez, verifica la solicitud y la reenvía al servidor de destino. Una vez que el servidor responde, el proxy recibe la respuesta y la devuelve al cliente original. Este proceso se realiza de manera transparente para el cliente, que puede acceder al contenido sin ser consciente de la intervención del proxy.

Existen diferentes tipos de proxies, cada uno con sus propias características y usos. Los más comunes son:

- **Proxy Web (Forward Proxy):** Permite el acceso a páginas web y es utilizado para mejorar la privacidad, el rendimiento o el filtrado de contenido.
- **Proxy Inverso (Reverse Proxy):** Se sitúa entre los servidores web y los usuarios finales. Ayuda a mejorar el rendimiento al almacenar en caché el contenido y distribuir la carga entre varios servidores.
- **Proxy Intermedio (o Proxy Transparente):** Opera sin requerir ninguna configuración en el cliente. Los usuarios no son conscientes de su existencia y sus solicitudes se envían automáticamente a través del proxy.

La figura 1.6 muestra una configuración típica de proxys para ofrecer a los usuarios un servicio web utilizando varios servidores físicos o virtuales (a través de la tecnología de los contenedores), en la cual se consigue mejorar el rendimiento y equilibrar la carga en los servidores.

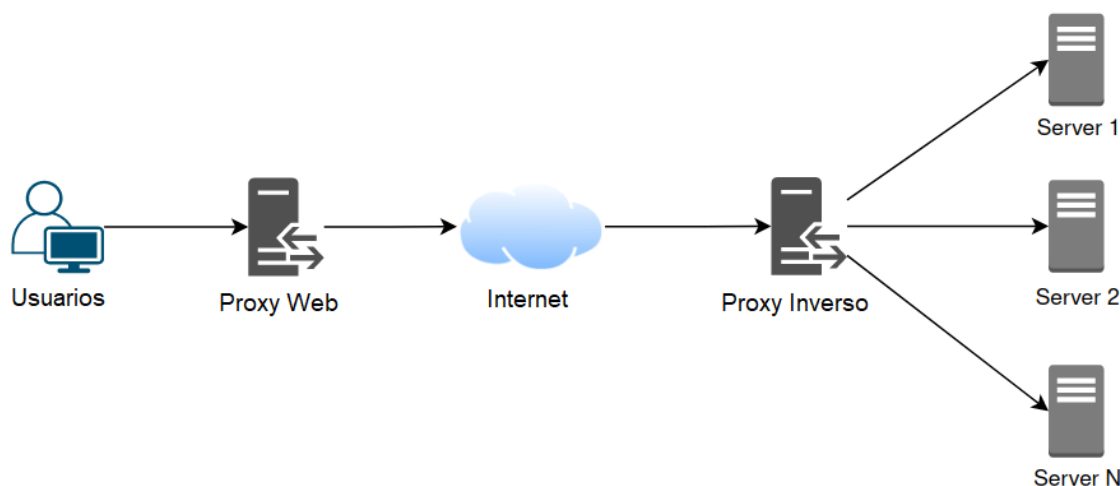


Figura 1.6: Diagrama de Proxys

El uso de un proxy ofrece varias ventajas. En primer lugar oculta la dirección IP real del cliente. Esto puede ser útil para proteger la identidad y la privacidad del usuario, como también para acceder a recursos restringidos geográficamente o para bloquear o permitir el acceso a ciertos tipos de contenido según una política predefinida. Además pueden almacenar en caché el contenido de los servidores web, lo que acelera la entrega de recursos a los clientes. Al distribuir la carga entre varios servidores, también pueden ayudar a reducir la carga en un solo servidor, mejorando el rendimiento global del sistema. Este último caso, se emplea en el presente trabajo a través de los subdominios que redirigen el tráfico al servicio al que se desea acceder.

2. Requerimientos

Este capítulo presenta los requerimientos propuestos para el Proyecto Integrador, presentados de manera clara y concisa.

2.1. Objetivos

2.1.1. Objetivo General

Implementar prácticas continuas seguras de la Ingeniería de Software para mejorar los procesos de desarrollo en proyectos IoT, en una aplicación de monitoreo de condiciones aptas para cultivo.

2.1.2. Objetivos Parciales

1. Investigar, seleccionar y planificar el despliegue de las herramientas apropiadas para construir una aplicación IoT con técnicas DevOps.
2. Diseñar y priorizar los requerimientos funcionales de una aplicación IoT para sensor temperatura, humedad y luminosidad, enviando los datos a un Broker MQTT.
3. Construir un prototipo de la aplicación para ESP32 y sus sensores.
4. Aplicar las prácticas continuas DevOps al proyecto: Integración Continua (CI), Tests automáticos, Actualizaciones por WiFi.
5. Integrar herramientas y prácticas para garantizar y verificar la seguridad del prototipo.
6. Evaluar los resultados obtenidos.
7. Redacción del informe y documentación del repositorio.

2.2. Usuarios

Este proyecto va dirigido tanto a desarrolladores como a usuarios finales, que llamaremos clientes. A continuación se distinguen y clarifican los aspectos orientados

a cada tipo de usuario.

2.2.1. Clientes

Los usuarios finales o clientes, son aquellas personas no necesariamente expertas en el ámbito IoT, que desean monitorear temperatura, humedad y luminosidad en un espacio reducido, el cual debe disponer de conectividad inalámbrica, idealmente en un entorno hogareño donde se pueda acceder fácilmente con un teléfono celular o smartphone para modificar las configuraciones del dispositivo en caso de ser necesario.

Además el smartphone se necesita para acceder al tablero o dashboard online donde se visualizan las métricas obtenidas por los sensores y se reciben alertas de que uno de los valores se encuentra fuera de los valores aceptables. Es vital ofrecerle al usuario un sistema confiable y seguro que almacene sus credenciales de forma confidencial y que reciba actualizaciones automáticas sin que el mismo deba efectuar cambios.

2.2.2. Desarrolladores

Este proyecto a su vez se pensó para facilitar el trabajo de los desarrolladores de proyectos IoT. Si bien en este proyecto se monitorean variables ambientales, las prácticas continuas y herramientas de análisis de seguridad pueden aplicarse a proyectos IoT con otras finalidades y la idea es precisamente construir un repositorio claro y ordenado de fácil adopción para otros desarrolladores.

2.3. Características y Requerimientos del Sistema

2.3.1. Requerimientos Funcionales

- **RF1:** La aplicación debe soportar sensores para medir temperatura ambiental, en el rango de 0°C a 40°C; humedad, entre 0 % (seco) y 100 % (agua); y luminosidad, entre 0 % (totalmente oscuro) y 100 % (día soleado).
- **RF2:** El dispositivo debe conectarse a internet por WiFi.
- **RF3:** Se debe disponer un servidor central para recibir, procesar y mostrar en una página web las métricas ambientales recolectadas por los sensores mencionados anteriormente.
- **RF4:** El dispositivo debe recibir actualizaciones automáticas de firmware por WiFi.

- **RF5:** Debe incluirse al menos una herramienta de análisis estático de código, que verifique en las bases de datos de vulnerabilidades conocidas, es decir, en la CVE o en la CWE.
- **RF6:** Cuando se tenga una nueva versión de firmware, se debe compilar y publicar la misma en el servidor de forma automática.
- **RF7:** El flujo de trabajo debe incluir una ejecución de pruebas automatizadas en un dispositivo real dentro de un ambiente de pruebas.

2.3.2. Requerimientos No Funcionales

- **RNF1:** El dispositivo debe emplear un uso eficiente de la energía, con un consumo no mayor a 400 mA.
- **RNF2:** El dispositivo debe ser pequeño y liviano, para que una persona pueda manipularlo fácilmente. El tamaño no debe exceder los 30cm de largo por 30cm ancho por 20cm de alto y el peso debe ser inferior a 500 gramos.
- **RNF3:** Todo el sistema del servidor debe ser replicable. Se debe incluir un instalador para instanciar y configurar los componentes necesarios.
- **RNF4:** El dispositivo debe ser escalable, para que lo usen al menos 3 clientes, cada uno pudiendo usarlo en su red WiFi hogareña.
- **RNF5:** El servidor debe ser compatible con al menos 3 distribuciones de Linux.

2.4. Topología

Se debe construir una topología similar a la de la figura [2.1](#).

En la figura se puede observar una red empresarial llamada Red Power-Pot (señalada en color rosado) en la cual se encuentra el servidor que da soporte a la aplicación junto a una placa de pruebas conectada al mismo.

La conexión con los usuarios finales se logra a través de internet, y cada uno de los mismos puede tener en su hogar uno o varios dispositivos conectados para sensor las variables ambientales.

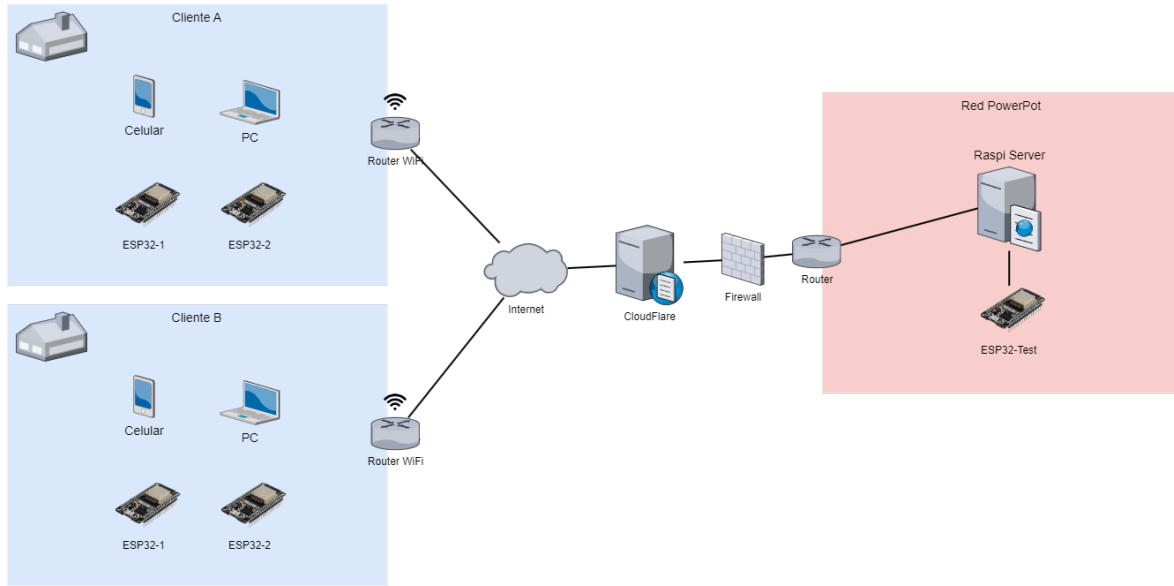


Figura 2.1: Topología

2.5. Análisis de Riesgos

Se realiza un análisis exhaustivo de cada uno de los requerimientos propuestos, señalando sus características y asignando un nivel de importancia para cada aspecto según de que requerimiento se trate.

Aspectos a tener en cuenta:

1. **Dificultad de Implementación (DI):** Se valora cada uno de los requerimientos en función de su dificultad de implementación, teniendo en cuenta tanto dificultades técnicas, como disponibilidad de herramientas y tiempo de llevar a cabo su implementación.
2. **Dependencia con Requerimientos (DR):** Existen requerimientos que dependen de que otros hayan sido implementados anteriormente. Otra posibilidad es que aunque no sea estrictamente necesario una implementación previa, podría ser recomendable dada la naturaleza del requerimiento.
3. **Impacto en el Servicio y su Desarrollo (ISD):** El impacto que un requerimiento tiene en el prototipo final tiene una mayor o menor valoración en función de si aporta mejoras significativas tanto al usuario como a los desarrolladores que trabajan en el ecosistema y en actualizaciones futuras.

A partir del análisis de cada uno de los requerimientos, se construye la tabla 2.1 en la que se pondera cada uno de ellos según los aspectos mencionados con los valores

alto, medio o bajo.

Requerimiento	DI	DR	ISD
RF1	Bajo	Alto	Alto
RF2	Bajo	Alto	Alto
RF3	Alto	Alto	Alto
RF4	Alto	Bajo	Alto
RF5	Bajo	Medio	Alto
RF6	Bajo	Alto	Alto
RF7	Bajo	Bajo	Bajo
RNF1	Bajo	Bajo	Medio
RNF2	Medio	Bajo	Medio
RNF3	Alto	Medio	Alto
RNF4	Alto	Medio	Medio
RNF5	Alto	Medio	Medio

Tabla 2.1: Tabla de Análisis de Requerimientos

2.6. Plan de Implementación

Luego de analizar la tabla, se elabora un plan de implementación para llevar a la práctica los prototipos en el orden adecuado según los aspectos de cada requerimiento. Se trata de un plan de prototipos iterativos incrementales a los cuales se les añade las características seleccionadas en función del análisis de los requerimientos.

2.6.1. Prototipo 1: Envío de datos al Broker MQTT

En este primer prototipo, se busca lograr un envío de datos entre el dispositivo ESP32 y el servidor central. Para este propósito se emplea el Broker MQTT, se construyen los tópicos y se envían datos en forma de simulacro con una aplicación de celular. Una vez verificada la correcta recepción de los datos, se realiza la implementación y configuración de los sensores que miden temperatura, humedad y luminosidad. Para lograr la comunicación entre el dispositivo y el broker, se requiere conexión inalámbrica o WiFi, por lo que se debe implementar las librerías necesarias para lograr la conexión. Además, se plantea un código simple para cargar un programa en la placa ESP32 para probar los sensores y las métricas que estos envían al broker.

En el prototipo 1 se implementan los requerimientos:

- **RF1:** La aplicación debe soportar sensores para medir temperatura ambiental, en el rango de 0°C a 40°C; humedad, entre 0 % (seco) y 100 % (agua); y luminosidad, entre 0 % (totalmente oscuro) y 100 % (día soleado).
- **RF2:** El dispositivo debe conectarse a internet por WiFi.
- **RF3:** Se debe disponer un servidor central para recibir, procesar y mostrar en una página web las métricas ambientales recolectadas por los sensores mencionados anteriormente.
- **RNF1:** El dispositivo debe emplear un uso eficiente de la energía, con un consumo no mayor a 400 mA.
- **RNF2:** El dispositivo debe ser pequeño y liviano, para que una persona pueda manipularlo fácilmente. El tamaño no debe exceder los 30cm de largo por 30cm ancho por 20cm de alto y el peso debe ser inferior a 500 gramos.

2.6.2. Prototipo 2: Flujo de Trabajo

Para el segundo prototipo se pretende construir un primer flujo de trabajo o workflow, para realizar la compilación del programa en la nube. Se busca añadir también un analizador de código para detectar posibles vulnerabilidades en etapas tempranas del desarrollo de software. Se implementa además un tablero o dashboard para visualizar las métricas obtenidas por los sensores de manera clara y ordenada.

En el prototipo 2 se implementan los requerimientos:

- **RF5:** Debe incluirse al menos una herramienta de análisis estático de código, que verifique en las bases de datos de vulnerabilidades conocidas, es decir, en la CVE o en la CWE.

- **RF6:** Cuando se tenga una nueva versión de firmware, se debe compilar y publicar la misma en el servidor de forma automática.

2.6.3. Prototipo 3: Ecosistema de Servicios en Raspberry Pi

En este prototipo se toma el servidor central que en principio sólo actuaba como Broker MQTT, y se le añaden los siguientes servicios:

- **Runner Self-Hosted:** Se añade una instancia local para ejecutar las tareas del workflow y tener acceso a la placa para cargarle el código.
- **Servidor Web:** Para mostrar el tablero o dashboard con las métricas a través de un sitio web al público.
- **Base de Datos:** Para almacenar las métricas obtenidas, lo cual tiene varios usos, entre ellos, promediar para evitar mostrar datos atípicos.
- **Servicio de Monitoreo de Hardware:** Resulta ideal tener un servicio que ayude a monitorear el estado de los recursos del servidor en tiempo real, incluyendo porcentaje de uso del CPU, memoria RAM y temperatura.
- **Administrador de Contenedores:** Ya que se planea implementar contenedores de Docker en prototipos futuros, resulta ideal instalar un administrador de contenedores, el cual permite ver los contenedores activos, detenerlos, reiniciar aquellos con fallos, entre otras opciones a través de una interfaz web que resulta muy práctica y fácil de usar para los desarrolladores.

En el prototipo 3 se implementan los requerimientos:

- **RF3:** Se debe disponer un servidor central para recibir, procesar y mostrar en una página web las métricas ambientales recolectadas por los sensores mencionados anteriormente.
- **RNF5:** El servidor debe ser compatible con al menos 3 distribuciones de Linux.

2.6.4. Prototipo 4: Docker y Testing

El cuarto prototipo tiene por objetivo añadir la tecnología de los contenedores de Docker al proyecto, para lograr empaquetar las dependencias de las aplicaciones y servicios a ejecutar y de esta manera construir un entorno fácilmente actualizable y replicable tanto por otros desarrolladores como si fuera necesario cambiar el servidor en el cual corren los servicios. En concreto, se construye una imagen de Docker para el runner, un archivo docker compose para instanciar el contenedor automáticamente con

un simple comando. Se añade además la posibilidad de hacer tests automáticos en una placa ESP32 antes de cargar el binario definitivo, cargándolos por cable USB gracias al runner self-hosted.

En el prototipo 4 se implementan los requerimientos:

- **RF6:** Cuando se tenga una nueva versión de firmware, se debe compilar y publicar la misma en el servidor de forma automática.
- **RF7:** El flujo de trabajo debe incluir una ejecución de pruebas automatizadas en un dispositivo real dentro de un ambiente de pruebas.
- **RNF3:** Todo el sistema del servidor debe ser replicable. Se debe incluir un instalador para instanciar y configurar los componentes necesarios.
- **RNF5:** El servidor debe ser compatible con al menos 3 distribuciones de Linux.

2.6.5. Prototipo 5: Actualizar por WiFi

Para el quinto prototipo se propone implementar la actualización del dispositivo ESP32 a través de WiFi, es decir, sin necesidad de que el mismo se encuentre conectado por cable USB, sino que reciba el binario por aire y lo instale, para luego reiniciarse y funcionar con la nueva versión del programa. En este prototipo se añadirá otro analizador de código diferente para tener otra perspectiva de seguridad.

En el prototipo 5 se implementan los requerimientos:

- **RF4:** El dispositivo debe recibir actualizaciones automáticas de firmware por WiFi.
- **RF5:** Debe incluirse al menos una herramienta de análisis estático de código, que verifique en las bases de datos de vulnerabilidades conocidas, es decir, en la CVE o en la CWE.

2.6.6. Prototipo 6: Actualizaciones Automáticas por HTTP

En este sexto y último prototipo, se busca mejorar el proceso de actualización de los dispositivos ESP32, implementando un servidor web que tiene siempre disponible la última versión del binario compilado y listo. Las placas ESP32 deben consultar frecuentemente al servidor para comprobar si tienen la misma versión o si ya hay una nueva que deben descargar e instalar. Este modelo es escalable a muchos dispositivos. Se debe añadir la posibilidad de que el usuario gestione sus credenciales WiFi desde su teléfono celular.

En el prototipo 6 se implementan los requerimientos:

- **RF4:** El dispositivo debe recibir actualizaciones automáticas de firmware por WiFi.
- **RNF4:** El dispositivo debe ser escalable, para que lo usen al menos 3 clientes, cada uno pudiendo usarlo en su red WiFi hogareña.

2.7. Esquema de Ramas GitFlow

Para llevar a cabo el desarrollo del proyecto se sigue el siguiente esquema de ramas, también llamado GitFlow, el cual se ilustra en la figura 2.2.

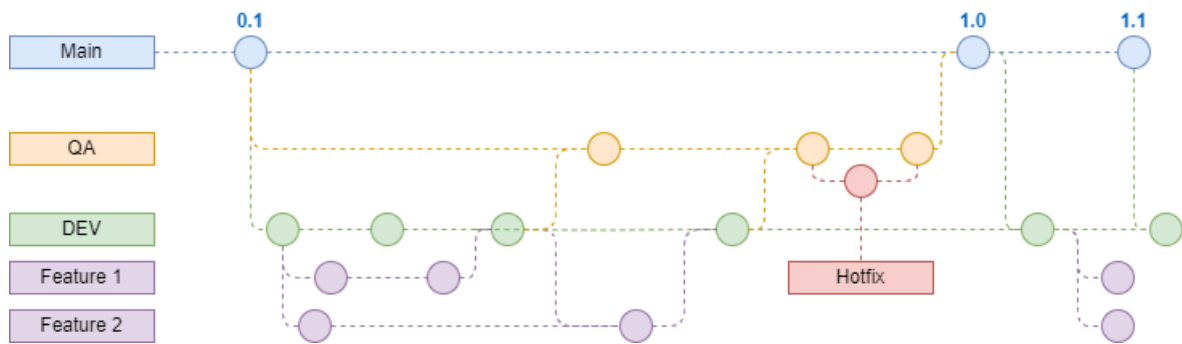


Figura 2.2: Esquema de Ramas GitFlow

En el esquema se observan las ramas que debe tener el repositorio principal de desarrollo de software del presente Proyecto Integrador. A continuación se describe cada una de las ramas presentadas:

- **Main:** Es la **rama principal**, también considerada **rama de producción**, la cual debe tener la última versión del código del programa que es la que usan todos los clientes. No debe tener problemas ni brechas de seguridad. No se puede subir código con push directamente en esta rama, la única manera de agregarlo es combinando con merge desde la rama QA.
- **QA (Quality Assurance):** Es la **rama de control de calidad**, contiene la misma versión que la rama principal, y su objetivo es disponer un entorno similar a producción en el cual efectuar pruebas de último minuto antes de que se lance una nueva versión al público.
- **Hotfix:** Se denomina hotfix a un cambio rápido destinado a solucionar un problema de último minuto.

- **DEV (Development):** Esta es la **rama de desarrollo**, en la cual se integran las características actualmente en desarrollo por el equipo de trabajo. Las pruebas se realizan sobre el código que se encuentra en esta rama para solucionar los errores antes de pasar a la instancia de QA. A partir de esta rama se bifurcan otras ramas cada una correspondiente a una característica a desarrollar, por ejemplo Feature 1 y 2 en la figura [2.2](#).

3. Selección de los Componentes

Antes de comenzar el desarrollo del proyecto resultó fundamental recavar información para seleccionar los componentes de hardware y software apropiados para trabajar adecuadamente. Las principales herramientas que se seleccionaron fueron:

- Dispositivo IoT o Microcontrolador
- Sensores
- Hardware del Servidor
- Plataforma de Control de Versiones
- Analizadores de Código

A continuación se presentan subsecciones con información relevante al momento de la selección de las herramientas.

3.1. Comparación de Dispositivos IoT

Una de las primeras consideraciones que se tuvo en cuenta a la hora de llevar a cabo un proyecto IoT es elegir el dispositivo que se usa, para asegurarse de que cumple con los requerimientos del proyecto y que su costo no es demasiado alto como para volverlo inviable económicamente.

Es importante aclarar que los **microcontroladores** se implementan en dispositivos que se conocen como **placas de desarrollo** (o development boards), que además del CPU incluyen pines de propósito general (o GPIO) y circuitos integrados para soportar periféricos como antena WiFi, puerto UART o puerto USB.

En el presente apartado, se presenta una comparativa entre las siguientes placas de desarrollo:

- NodeMCU ESP8266 [15]
- NodeMCU ESP32S [14]
- Raspberry Pi Pico W [38]

Se muestran imágenes de cada una de las placas de desarrollo mencionadas en las figuras 3.1, 3.2, y 3.3.

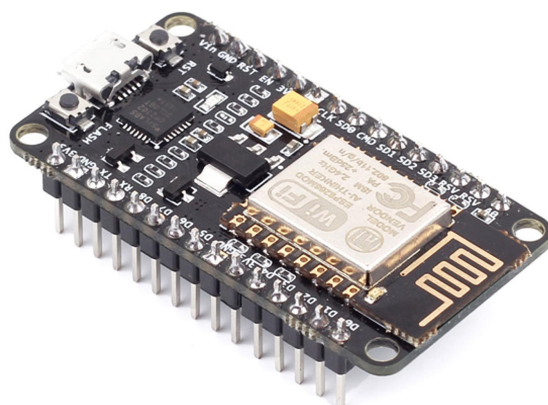


Figura 3.1: Placa NodeMCU8266



Figura 3.2: Placa NodeMCU32S

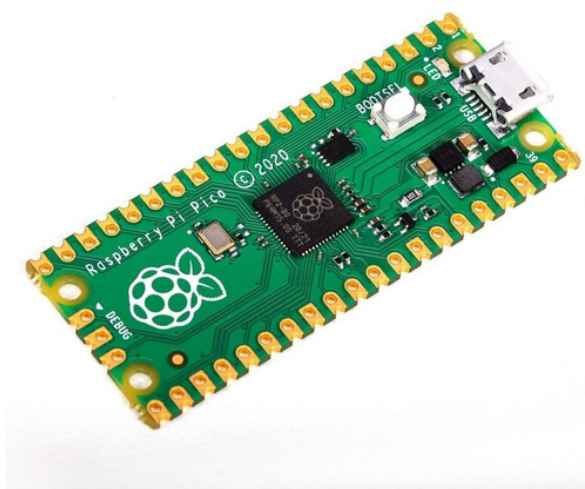


Figura 3.3: Placa Raspberry Pi Pico W

El primer aspecto que se consideró en esta comparativa es el rendimiento de cada placa de desarrollo, el cual se midió con la herramienta **Arduino Speed Test** [46]. Con la misma se hicieron las siguientes pruebas:

Test NOP: Determina el tiempo de sobrecoste (o overhead) que dedica el procesador en la llamada a una operación. Para ello, se hará uso de llamadas a una **no operation** (o nop), cuyo tiempo de proceso corresponde exactamente a un ciclo de reloj (el inverso de la frecuencia).

	Raspi Pico	ESP8266	ESP32
Frecuencia [MHz]	133	80	240
Sobrecoste [us]	0,034	0,050	0,022

Tabla 3.1: Test NOP

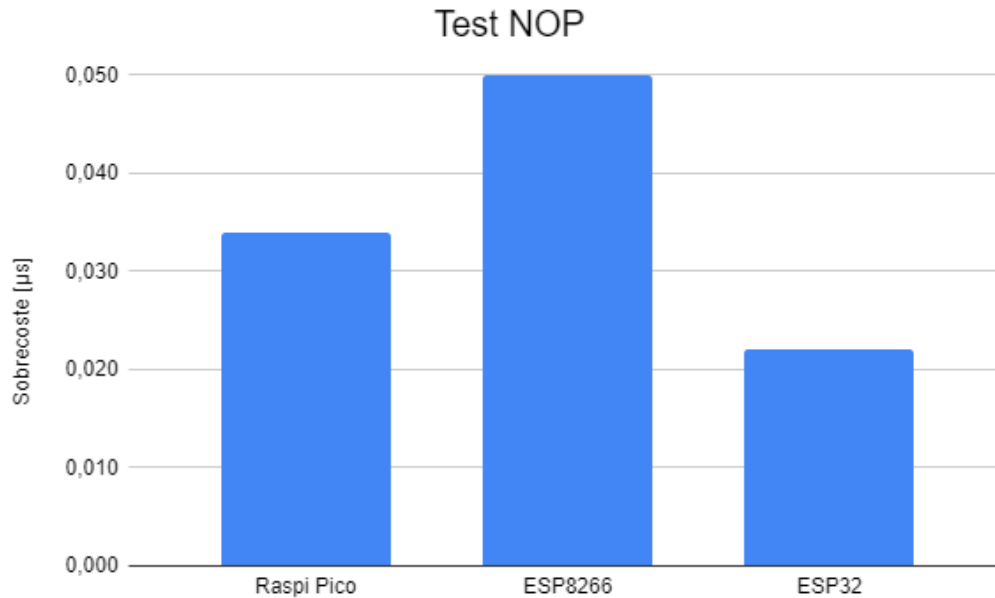


Figura 3.4: Test NOP

Test de pines digitales: Se midió el tiempo que se tarda en realizar operaciones de lectura, escritura y cambio de estado en los pines digitales.

	Raspi Pico	ESP8266	ESP32
digitalRead [us]	0,022	0,937	0,141
digitalWrite [us]	0,278	0,87	0,109
pinMode [us]	0,866	1,955	2,506

Tabla 3.2: Test de pines digitales

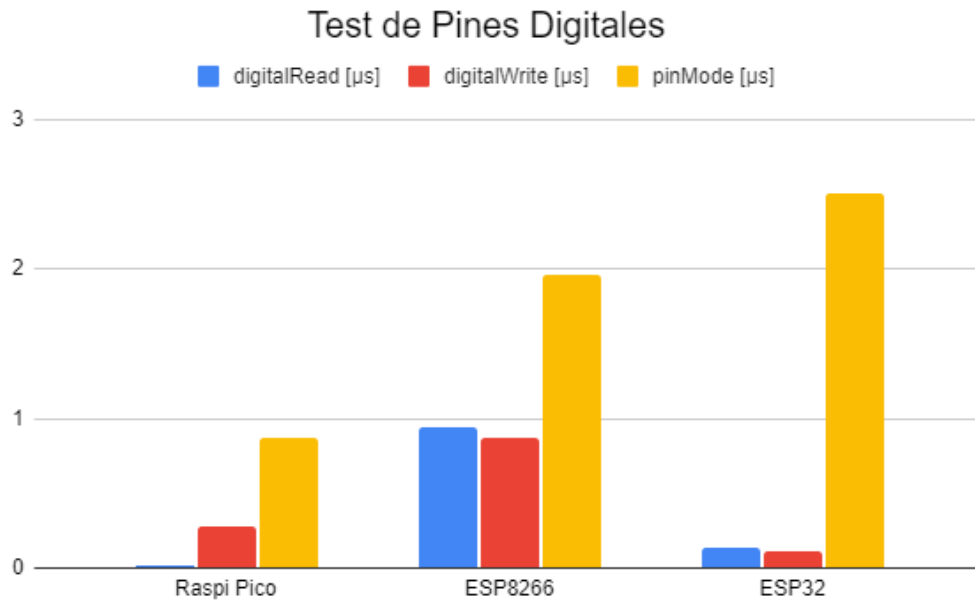


Figura 3.5: Test de pines digitales

Test de operaciones con números enteros: Se midió el tiempo en realizar operaciones con números enteros (integer).

	Raspi Pico	ESP8266	ESP32
multiply integer [us]	0,057	0,148	0,053
divide integer [us]	0,228	0,432	0,059
add integer [us]	0,056	0,135	0,053

Tabla 3.3: Test de operaciones con enteros

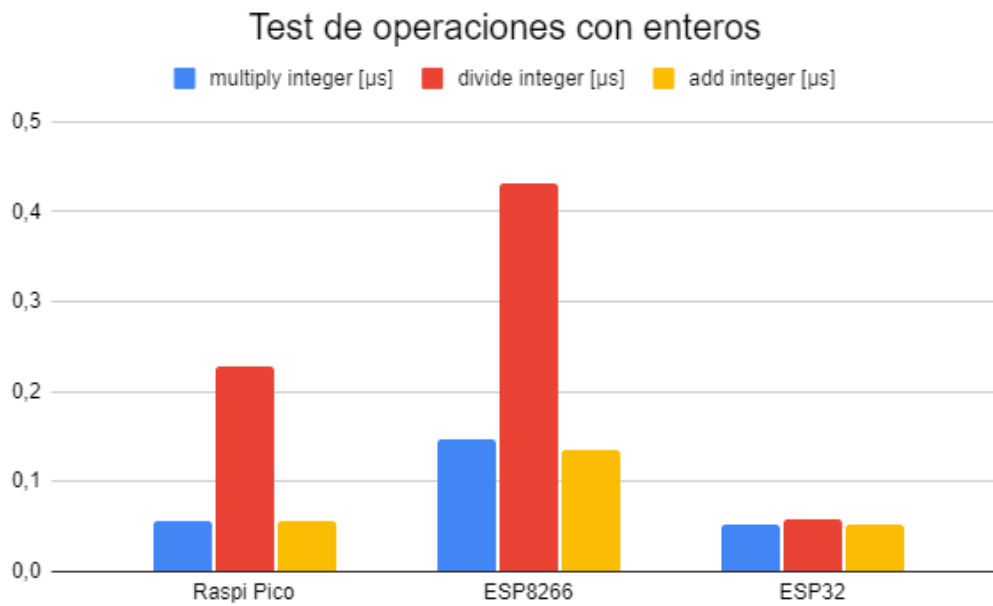


Figura 3.6: Test de operaciones con enteros

Test de operaciones con números flotantes: Se midió el tiempo en realizar operaciones con números de punto flotante (o float).

	Raspi Pico	ESP8266	ESP32
multiply float [us]	0,583	0,738	0,054
divide float [us]	0,473	3,772	0,224
add float [us]	0,688	0,683	0,054

Tabla 3.4: Test de operaciones con flotantes

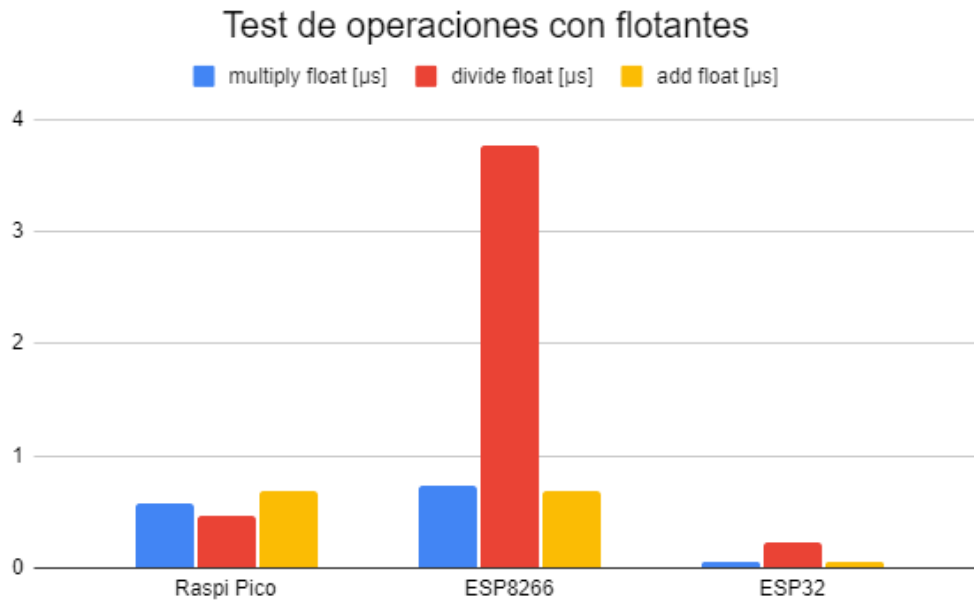


Figura 3.7: Test de operaciones con flotantes

Otras características: a continuación se muestra la tabla 3.5 en la que se comparan los periféricos de las placas mencionadas y otras características de interés como la conectividad y el precio.

	Raspi Pico	ESP8266	ESP32
Pines GPIO	30	16	36
ADC	3 x 12 bits	1 x 10 bits	18 x 12 bits
UART	2	2	3
SPI	2	2	4
I2C	2	1	2
PWM	16 x 16 bits	4 x 14 bits	16 x 16 bits
WiFi	802.11 b/g/n 2.4GHz	802.11 b/g/n 2.4GHz	802.11 b/g/n 2.4GHz
Bluetooth	No	No	BT 4.2 y BT LE
Deep Sleep	18 [uA]	10 [uA]	10 [uA]
Precio	6 USD	5 USD	5 USD

Tabla 3.5: Otras características

En conclusión, **se optó por trabajar con la placa de desarrollo NodeM-CU32s que hace uso del microcontrolador ESP32** porque presenta mejor rendimiento general [42], un precio similar y además es una placa muy popular en el sector Internet of Things, lo que facilita la implementación de las aplicaciones gracias a la amplia variedad de librerías disponibles y ante los problemas es más fácil encontrar

ayuda en la comunidad.

3.2. Selección de Sensores

Para medir la **temperatura** y la **humedad del aire** se emplea un **sensor DHT11** [1], porque es un sensor económico, compacto y permite medir 2 magnitudes, lo que resulta ideal para mantener simple el prototipo. Presenta una señal digital de salida de 8 bits de resolución y posee un rango de temperatura de trabajo entre 0º y 50ºC. Su encapsulado es pequeño y con 4 pines, 2 para alimentación y los otros 2 para las señales de salida que representan temperatura y humedad.

Existe una alternativa y es el **sensor DHT22** [2], el cual es similar en principio de funcionamiento y pines de entrada salida, pero con una tolerancia menor a la hora de entregar mediciones, a un precio de alrededor del triple del precio del sensor DHT11. En este proyecto no se requieren mediciones exactas sino más bien presentar un prototipo real sobre el cual aplicar las prácticas continuas en las que se basa el proyecto. Por estos motivos se seleccionó el sensor DHT11 para medir las magnitudes de temperatura y humedad del aire.

A continuación se presenta una imagen del sensor DHT11 en la figura 3.8.

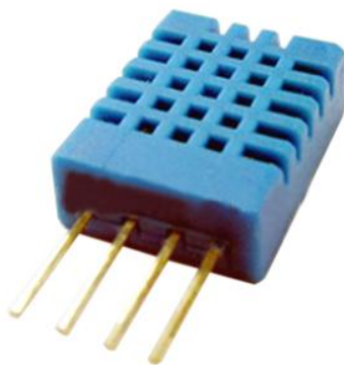


Figura 3.8: Sensor DHT11

Para medir la **luminosidad** del ambiente se emplea un **fotorresistor** o **LDR** por sus siglas en inglés (Light-Dependent Resistor), el cual consiste en un resistor con una **célula fotorreceptora** que cambia su valor resistivo según la intensidad lumínica que incide sobre la célula. El valor de resistencia eléctrica de un LDR es bajo cuando hay luz incidiendo en él (puede descender hasta 50 ohms) y muy alto cuando está a oscuras (varios megaohms).

En concreto, se escogió un **módulo** llamado **Light-Sensor**[41] de marca Duai-tek, popularmente conocido como módulo LDR Arduino, debido a que este microcon-

trolador es ampliamente utilizado para proyectos de electrónica. Este módulo agrupa el fotorresistor LDR, un potenciómetro para calibrar la sensibilidad, un comparador, LEDs indicadores y 4 pines que se describen a continuación.

1. **VCC**: Voltaje de Alimentación (3.3V - 5V DC)
2. **GND**: Terminal común o tierra
3. **DO**: Digital Output o salida digital
4. **AO**: Analog Output o salida analógica

La señal digital de salida se obtiene gracias a la comparación de la señal analógica del LDR con el potenciómetro que brinda un voltaje que actúa como valor de umbral (o threshold), consiguiendo una salida digital alta o baja según el resultado de la comparación. El módulo funciona con un voltaje de entrada de entre 3,3V a 5V. A continuación se presenta la figura 3.9 donde se puede ver el módulo con sus respectivos componentes.

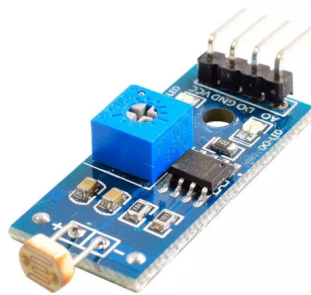


Figura 3.9: Módulo Sensor de Luz

Para medir la **humedad del suelo** se utiliza un **sensor de humedad de suelo capacitivo v1.2** [40], conocido por su nombre en inglés **Capacitive Soil Moisture Sensor v1.2**. El funcionamiento del sensor se basa en medir la **capacitancia** entre 2 electrodos insertados dentro del suelo, la cual dependerá de la humedad del suelo, por lo que para un suelo muy húmedo tendremos una capacitancia muy baja y para un suelo muy seco la capacitancia será muy alta. El electrodo va conectado a una tarjeta de acondicionamiento que entrega una salida analógica. La salida analógica (AOUT, Analog Output) entrega un voltaje analógico desde 0V para un suelo muy húmedo hasta 5V para un suelo muy seco. Este sensor posee 3 pines, que se describen a continuación:

1. **GND**: Terminal común o tierra
2. **VCC**: Voltaje de alimentación (3.3V - 5V DC)

3. AOUT: Analog Output o Salida analógica

La principal ventaja de usar este sensor en lugar de uno que mida la humedad por resistividad, es que previene la oxidación y la corrosión, que es muy habitual en sensores resistivos.

La figura 3.10 muestra el sensor empleado para medir humedad de suelo en el proyecto.



Figura 3.10: Sensor de Humedad de Suelo Capacitivo

3.3. Selección del Hardware para el Servidor

Además del microcontrolador que se utiliza para recolectar los datos, el proyecto requiere un servidor que ofrezca varios servicios, entre ellos:

- **Broker MQTT** para gestionar los tópicos que reciben los datos de los sensores.
- **Runner CI/CD** para ejecutar las tareas del flujo de trabajo.
- **Servidor NodeRED** para presentar en una interfaz web los datos de manera clara y ordenada.
- **Servidor de Actualizaciones de Firmware** para que las placas descarguen actualizaciones automáticas cuando estén disponibles.

3.3.1. Raspberry Pi 4 Model B

El dispositivo Raspberry Pi 4 Model B se trata de una pequeña computadora diseñada para ser económica y eficiente, sin dejar de proveer potencia computacional y estabilidad de funcionamiento. En la figura 3.11 se observa este dispositivo cubierto por un disipador de aluminio color negro con 2 ventiladores que se ocupan de refrigerar el CPU.



Figura 3.11: Raspberry Pi 4 Model B

A continuación se detallan las características de hardware de este dispositivo:

- Procesador Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz
- Memoria RAM de 8GB LPDDR4-3200 SDRAM
- Conectividad WiFi 2.4 GHz y 5.0 GHz IEEE 802.11ac, Bluetooth 5.0 y BLE
- 2 puertos USB 2.0 y 2 puertos USB 3.0
- 40 pines de GPIO
- 2 puertos micro-HDMI
- Slot para tarjeta de memoria MicroSD
- Conector USB-C para alimentación

Es importante destacar que los procesadores Cortex pertenecen a la arquitectura del conjunto de instrucciones **ARM (Advanced RISC Machines)**, que emplean un conjunto de instrucciones de procesador diferente al de la arquitectura x86 que normalmente se tiene en equipos de escritorio y notebooks, por lo tanto, las aplicaciones de la arquitectura x86 no funcionarán en ARM y viceversa, lo cual se debe tener en consideración a la hora de elegirlo como servidor y asegurarse de que las aplicaciones necesarias tengan su versión para sistemas ARM. Para este proyecto, esta limitación no supone un problema, ya que se buscaron aplicaciones compatibles con la arquitectura ARM que emplea este dispositivo.

Este dispositivo resulta de utilidad porque puede ejecutar los servicios mencionados, es pequeño, liviano y de fácil mantenimiento, siendo muy accesible para pequeños

proyectos. Es por estos motivos que **se seleccionó este dispositivo para funcionar como servidor**. Más adelante se detallarán los servicios que corren en este servidor junto con su respectiva información de referencia.

3.4. Plataforma de Control de Versiones

Existen numerosas plataformas en línea que ofrecen alojamiento para repositorios de Git, dentro de las más populares se encuentran **GitHub**, **GitLab** y **Bitbucket**. Es importante distinguir entre **Git**, el sistema que permite versionar el proyecto, y las respectivas plataformas que permiten subir esta información del versionado a la nube para que todos los desarrolladores se mantengan actualizados. A continuación se describen brevemente las 3 plataformas más populares antes mencionadas.

GitHub es una plataforma de alojamiento de repositorios de código basada en Git que ofrece una amplia variedad de herramientas para el desarrollo de software y la colaboración entre equipos. Es popular gracias a su amplia comunidad y su facilidad de uso, y es utilizado por organizaciones de todos los tamaños.

GitLab es una plataforma de gestión de código y proyectos de software que también se basa en Git. Ofrece características similares a GitHub, pero incluye herramientas adicionales para la gestión de proyectos y el seguimiento de problemas. GitLab también es muy utilizado en la industria, pero es más popular entre las grandes empresas y organizaciones gubernamentales.

Bitbucket es otra plataforma de alojamiento de repositorios de código basada en Git que ofrece una amplia variedad de herramientas para el desarrollo de software y la colaboración entre equipos. A diferencia de GitHub y GitLab, Bitbucket se enfoca más en el trabajo en equipo y ofrece características especialmente diseñadas para equipos más grandes, como integraciones con herramientas de gestión de proyectos y una opción de pago para organizaciones que deseen acceder a características adicionales.

Se presenta un cuadro comparativo de sus características principales:

Característica	GitHub	GitLab	Bitbucket
Tipo de plataforma	Comunidad	Privado	Privado
Alojamiento de código	Gratuito (2 GB) y pago (100 GB)	Gratuito (2 GB) y pago (200 GB)	Gratuito (2 GB) y pago (100 GB)
Repositorios públicos	Gratuitos	Gratuitos	Gratuitos
Repositorios privados	Gratuitos (1000) y pago (ilimitados)	Ilimitados	Gratuitos (5) y pago (ilimitados)
Integraciones	Más de 2000 integraciones	Más de 1000 integraciones	Más de 1000 integraciones
Herramientas de colaboración	Anotaciones, Issues, Pull Requests	Anotaciones, Issues, Pull Requests	Anotaciones, Issues, Pull Requests
Lenguajes soportados	Más de 200	Más de 200	Más de 200
Integración con CI y CD	Si (GitHub Actions)	Si (GitLab CI/CD)	Si (Bitbucket Pipelines)

Tabla 3.6: Comparativa de plataformas de alojamiento

Luego de realizar la comparativa entre las distintas plataformas online que ofrecen alojamiento para repositorios de control de versiones, se concluyó que las 3 plataformas cumplen adecuadamente la función de alojar el código de control de versiones del proyecto, sin embargo es importante destacar que **GitHub** resulta ser más cómoda para trabajar ya que es mucho más popular entre los proyectos de desarrollo de código abierto. Esto significa que tiene integraciones con muchas herramientas de desarrollo y además ofrece Actions (es decir, códigos con conjunto de comandos para tareas populares) creadas por la comunidad para agregar a los flujos de trabajo que se describen más adelante en este informe.

Dadas estas facilidades es por lo que **se optó por GitHub** como plataforma de alojamiento de código para el control de versiones del proyecto, y a crear flujos de trabajo con **GitHub Actions**.

3.5. Analizadores de Código

Los analizadores de código son herramientas de software que se encargan de analizar el código del proyecto y compararlo con bases de datos de vulnerabilidades como la CVE que se describe en la sección [Bases de Datos de Vulnerabilidades](#).

En el primer caso, los analizadores locales considerados fueron **Clang Tidy**[55], **Cppcheck**[13] y **PVS-Studio**[37], ya que estos 3 analizadores tienen integración con PlatformIO, y están soportados oficialmente, lo que significa que resulta sencillo incorporarlos para analizar el código de este Proyecto Integrador. PVS-Studio no resulta viable porque es una solución de pago y para usarse debe comprarse una licencia, motivo por el cual se consideran los dos primeros. Tanto Clang Tidy como Cppcheck ofrecen análisis estático de código C/C++ que permite identificar errores tales como memory leaks, uso de funciones deprecadas, buffer overflow, referencias a apuntadores nulos, entre otras debilidades que podrían introducirse al codificar.

Después de haber probado estas soluciones, **se consideró que Cppcheck** presenta una salida más clara y ordenada pudiendo además filtrar entre 3 niveles de prioridad, alto, medio y bajo.

Por otro lado, para analizar los commits que se añaden al repositorio en línea se tienen alternativas como **CodeQL** [23], **SonarCloud**[50], **SonarQube**[51] y **Coverity**[54]. A continuación se describen brevemente cada una de estas.

CodeQL es una herramienta desarrollada por GitHub que se utiliza para el análisis de seguridad de código y detección de vulnerabilidades en repositorios. Utiliza un lenguaje de consulta específico para el análisis de código, conocido como "QL" (Query

Language), que permite a los desarrolladores definir consultas para buscar patrones específicos en el código que puedan indicar posibles vulnerabilidades. Estas consultas pueden abarcar una amplia gama de problemas de seguridad, como inyecciones de SQL, desbordamientos de buffer, entre otros. CodeQL se ha vuelto especialmente útil en el proceso de desarrollo seguro de software y en la detección temprana de problemas de seguridad, ya que permite a los equipos de desarrollo abordar las vulnerabilidades antes de que se desplieguen en producción, lo que ayuda a prevenir ataques y brechas de seguridad.

SonarCloud es una plataforma en línea que proporciona servicios de análisis estático de código y evaluación continua de la calidad del código para proyectos de software. Su objetivo principal es ayudar a identificar y resolver problemas de calidad del código en etapas tempranas del ciclo de vida del desarrollo.

SonarCloud y **SonarQube** son productos de la misma empresa, **SonarSource**. Ambos funcionan de manera similar con algunas diferencias. SonarCloud es una plataforma en la nube, lo que significa que SonarSource hospeda y administra la infraestructura necesaria. Los proyectos se analizan en la nube y los resultados están disponibles en línea. Por otro lado, SonarQube es una solución que los equipos pueden implementar en sus propios servidores (self-hosted) o en la nube de su elección. Proporciona más flexibilidad en términos de alojamiento y control sobre la infraestructura.

Luego de probar estas soluciones, **se optó por utilizar Cppcheck, CodeQL y SonarQube en paralelo**, ya que estas se adaptan muy bien al proyecto. Sin embargo, es importante destacar que **bajo la licencia gratuita de SonarQube sólo se pueden analizar archivos de código en lenguaje Python** y no de lenguaje C++, ya que para analizar estos corresponde utilizar la versión que requiere una licencia de pago.

3.6. Compilación y Carga del programa

3.6.1. ESP-IDF

La compilación del programa depende del lenguaje de programación utilizado al codificar. En este proyecto se hace uso del **lenguaje C++**, ya que el mismo es una elección habitual para el desarrollo de proyectos IoT, porque se disponen de numerosas librerías que tienen implementadas funciones comunes a este tipo de trabajos.

CMake es un software libre y de código abierto multiplataforma para gestionar la automatización de la compilación del software utilizando un método independiente del

compilador. Soporta jerarquías de directorios y aplicaciones que dependen de múltiples bibliotecas [57].

El fabricante de la placa de desarrollo NodeMCU32s es la empresa **Espressif** [17], la cual proporciona una herramienta para la gestión de la carga del programa en la placa física llamada **ESP-IDF** (Espressif IoT Development Framework). Las herramientas de software necesarias para cargar el código en la placa son CMake y un ejecutable de código Python (llamado idf.py). En la figura 3.12 se puede ver el diagrama de las herramientas mencionadas, suministrado por el fabricante en su guía de inicio [16].

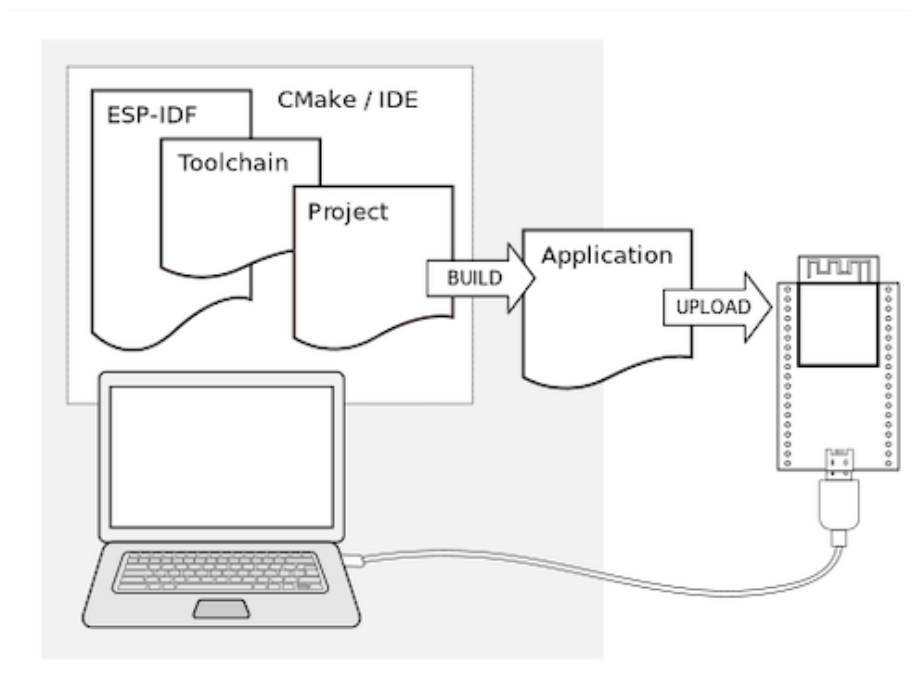


Figura 3.12: Proceso de carga del programa en la placa NodeMCU32s

Como se puede observar el hecho de **subir un programa al dispositivo físico resulta ser una tarea que demanda tiempo y esfuerzo**, ya que implica descargar e instalar el framework ESP-IDF, las librerías de funciones a utilizar, el compilador CMake, el intérprete de Python, y los controladores USB para acceder a los puertos a través de los cuales se carga el código en la placa. Es por este motivo que se buscó una alternativa para simplificar este proceso, la cual se presenta en la herramienta PlatformIO que se describe en los párrafos subsiguientes.

3.6.2. PlatformIO

PlatformIO[43] es una plataforma de desarrollo de código abierto que proporciona un entorno unificado para compilar y cargar programas en una amplia gama

de dispositivos IoT. Su enfoque multiplataforma y su capacidad de integración con diferentes frameworks, como Arduino y ESP-IDF, hacen de PlatformIO una elección popular entre los desarrolladores de IoT.

PlatformIO se destaca por su conjunto de funcionalidades que facilitan el desarrollo y la implementación de programas en dispositivos IoT. Permite la gestión sencilla de bibliotecas, brindando acceso a una gran variedad de librerías utilizadas en el ámbito de IoT. Además, ofrece una interfaz intuitiva que facilita la configuración de proyectos, la selección del hardware objetivo y la gestión de dependencias. PlatformIO también ofrece una amplia documentación y una comunidad activa, lo que agiliza la resolución de problemas y la adopción de nuevas funcionalidades. Además, su capacidad de integración con IDEs populares, como Visual Studio Code, potencia aún más su utilidad y flexibilidad. El equipo de trabajo usa Visual Studio Code como entorno de desarrollo agregando la extensión de PlatformIO para trabajar de manera cómoda y eficiente.

Dadas las ventajas y facilidades de uso de esta herramienta, se hace uso de ella para llevar a cabo **la compilación, configuración de bibliotecas y carga del programa** a los dispositivos IoT en el presente Proyecto Integrador. La figura 3.13 muestra una captura de pantalla de la extensión PlatformIO para el IDE Visual Studio Code, en su pantalla de bienvenida donde se ven las opciones de configuración para seleccionar el proyecto y el dispositivo para trabajar. Además, en la parte inferior se observa un conjunto de botones, señalado en color morado, que resultan prácticos a la hora de trabajar ya que se muestran en todo momento y permiten compilar, cargar el programa, ejecutar los tests, y ver el Serial Monitor con solo un click sin necesidad de escribir los comandos en la terminal.

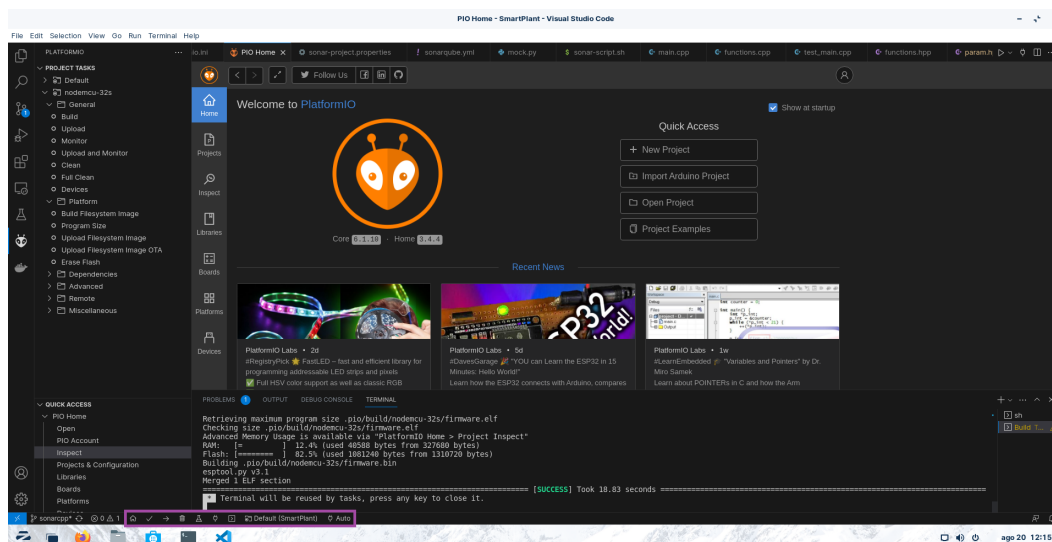


Figura 3.13: Captura de Pantalla de la extensión PlatformIO para VSCode

4. Actividades Desarrolladas

En este capítulo se incluye una descripción de las actividades desarrolladas como parte del Proyecto Integrador, describiendo los prototipos intermedios de manera iterativa, detallando las necesidades e inconvenientes enfrentados en cada etapa.

4.1. Preparación Preliminar

4.1.1. Investigación

En primer lugar se investigó sobre la cultura DevOps, las prácticas continuas que se mencionan en el [Marco Teórico](#), obteniendo la información a partir de documentación, artículos científicos a los cuales se hace referencia, y la lectura del libro ***The DevOps Handbook*** [32], que menciona cómo surgió esta cultura y cuáles son las problemáticas que viene a solucionar.

4.1.2. Configuración del Entorno

Una vez seleccionadas las herramientas para usar en los primeros prototipos, se procedió a configurar el entorno, lo que implica:

- Instalación del entorno de desarrollo Visual Studio Code, como también las herramientas esenciales para trabajar con lenguaje C++, Python y el framework Arduino.
- Instalación del sistema operativo Ubuntu Server 22.04 en su versión ARM para la Raspberry Pi 4.
- Configuración del servidor Raspberry Pi para acceder a él por protocolo SSH, configurando un puerto especial en el router, distinto del habitual 22 que se usa por defecto en este protocolo, para agregar una medida defensiva contra intentos de conexión por parte de bots de internet.

4.2. Prototipo 1: Envío de datos al Broker MQTT

4.2.1. La Aplicación

La aplicación consiste en un programa sencillo con el objetivo de monitorear variables ambientales y determinar si se está en condiciones aptas para cultivar o no. El conjunto de sensores seleccionados se ocupan de **medir temperatura, humedad del aire, humedad del suelo y luminosidad**, para dar aviso al usuario a través de un mensaje personalizado, como por ejemplo un mensaje de Telegram o un correo electrónico, en caso de que alguno de los valores sensados esté fuera de los valores aceptables y se deba intervenir, por ejemplo, regando las plantas para aumentar el nivel de humedad en el suelo.

Es importante mencionar que esta aplicación se mantiene sencilla para emplear los esfuerzos del equipo de trabajo en las prácticas DevOps que facilitan el desarrollo y permiten verificar y reforzar la seguridad durante el proceso de desarrollo de proyectos IoT. La idea es brindar un proyecto IoT que pueda manipular datos de cualquier naturaleza, pensado para facilitar el trabajo a otros desarrolladores que pretendan comenzar un proyecto IoT pudiendo tomar la estructura y prácticas continuas del presente Proyecto Integrador y agilizar el proceso de desarrollo.

4.2.2. Prototipo Físico

Luego de haber seleccionado los dispositivos y los sensores para trabajar, se construyó un primer prototipo utilizando una placa de pruebas protoboard, la placa ESP32 y los sensores. Este prototipo consiste básicamente en **recolectar los datos obtenidos por los sensores y enviarlos a través del protocolo MQTT al broker** ubicado en el servidor Raspberry Pi descrito en la sección [Raspberry Pi 4 Model B](#). En la figura 4.1 se visualiza una fotografía del prototipo armado junto a sus respectivos sensores.

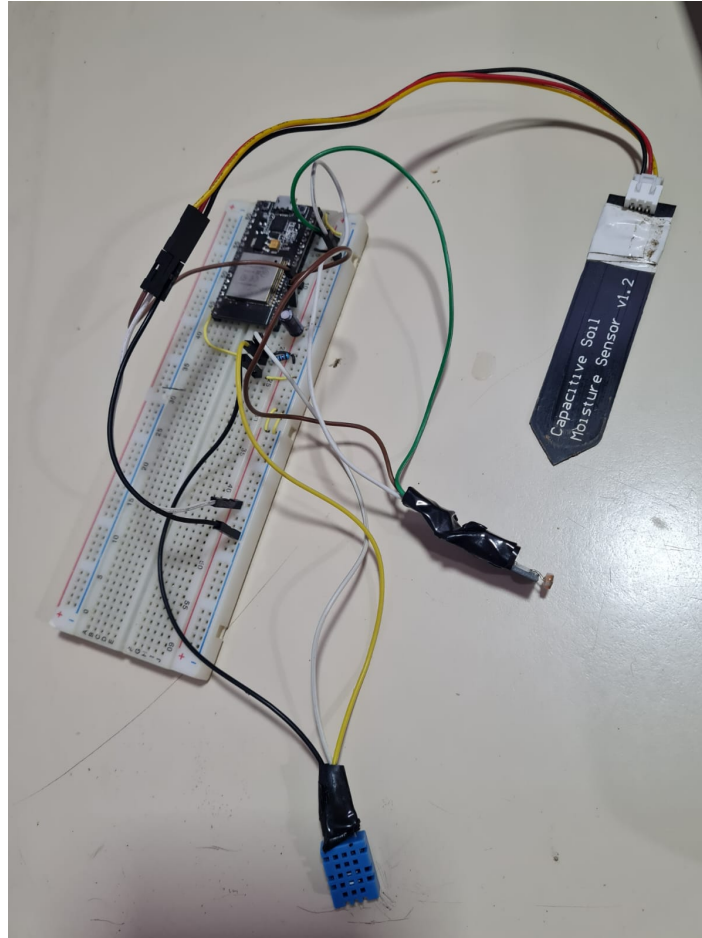


Figura 4.1: 1er prototipo montado en protoboard

Para lograr la conectividad del prototipo es necesario incluir la librería WiFi, junto a las credenciales de la red a la que se desea conectar, las cuales se suministran a través de variables de entorno para evitar colocarlas en el código. De este modo, el programa hace uso de la antena integrada para enviar los datos recavados a través de la red inalámbrica.

La compilación y carga del programa en la placa ESP32 se consigue gracias a la herramienta **PlatformIO** (véase la sección [Compilación y Carga del programa](#)), gracias a la cual se simplifica el proceso de descarga y gestión de las librerías propias del framework Arduino con el que se trabaja, además de compilar y subir el código al prototipo con un comando o click en el botón correspondiente.

4.2.3. MQTT Broker

El servicio utilizado para montar el Broker MQTT es **Eclipse Mosquitto** [22], el cual consiste en un Broker de mensajes de código abierto que implementa el protocolo

MQTT en sus versiones 5.0, 3.1.1 y 3.1. Consiste en un servicio liviano que puede ser ejecutado en computadoras de bajo consumo como las SBC (Single Board Computers) o en servidores clásicos. La ya mencionada Raspberry Pi 4 es un ejemplo de SBC.

Mosquitto ofrece una biblioteca de funciones en lenguaje C que se utilizan para la implementación de los clientes en las placas ESP32. Mosquitto además posee integración con el framework Arduino, el cual es reconocido por ser sencillo de usar y recomendado para estudiantes y aficionados al campo de los microcontroladores y proyectos IoT. Dado que el equipo de trabajo emplea el framework Arduino y este está soportado por la herramienta PlatformIO usada para compilar, Mosquitto resulta ser una solución adecuada por su robustez, simplicidad y además por ser de uso libre sin necesidad de pagar una licencia.

La instalación del servicio Mosquitto se debe realizar con el binario específico para sistemas operativos Linux con arquitectura ARM, mediante el gestor de paquetes **apt**. Este tipo de instalación se realiza directamente sobre el sistema operativo, lo cual significa que no se ha virtualizado un entorno específico ni tampoco un entorno contenerizado para esta aplicación, para hacer pruebas básicas con la instalación más limpia y simple posible.

A continuación se muestra una fotografía del servidor Raspberry Pi funcionando, conectado a la red por cable UTP (amarillo), y a la alimentación por cable USB tipo C (abajo). El cable adicional conectado al puerto USB que se observa a la derecha es un adaptador SATA a USB empleado para conectar un disco externo al equipo, ubicado por debajo del dispositivo dentro de la carcasa color azul.

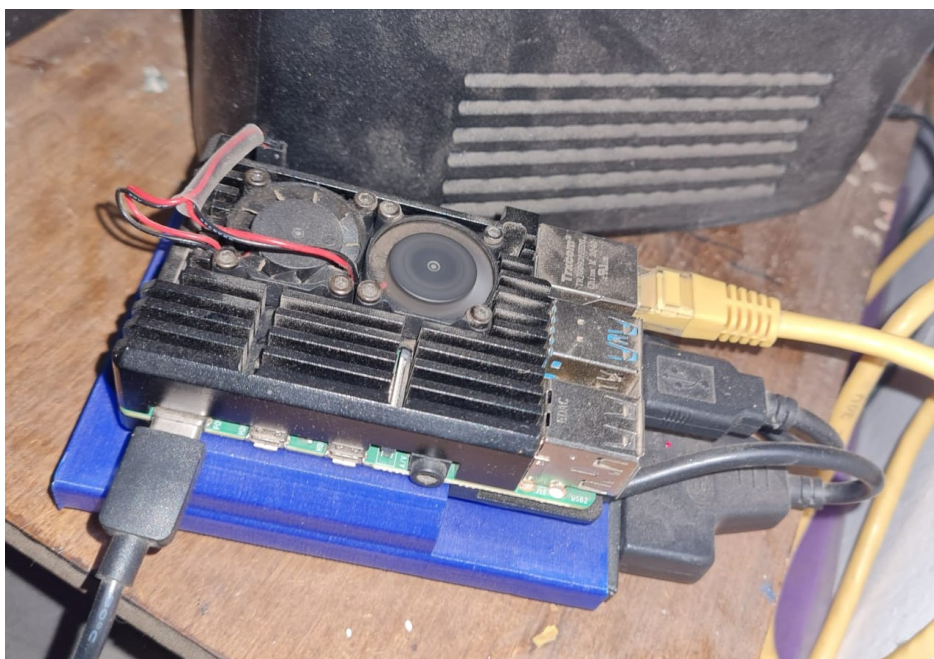


Figura 4.2: Servidor Raspberry Pi en funcionamiento

4.2.4. Tópicos MQTT

Se crearon 4 tópicos MQTT para escribir y leer la información recavada por los sensores, que se describen a continuación:

- **esp32/temperatura**: Datos del sensor de temperatura (DHT11)
- **esp32/humedad**: Datos del sensor de humedad de aire (DHT11)
- **esp32/humedadSuelo**: Datos del sensor de humedad del suelo (Sensor Capacitivo de Humedad de Suelo)
- **esp32/nivelLuz**: Datos del sensor de luminosidad (Modulo LDR)

Una vez contruidos los tópicos y con el servicio del Broker en funcionamiento, surge la necesidad de visualizar los datos transmitidos y probar el correcto funcionamiento del prototipo y sus sensores. Para este propósito se necesita encontrar la manera de enviar y recibir mensajes MQTT y ver su contenido. Para esto se usó temporalmente una aplicación de celular llamada **MQTT Dash**, que permite conectarse a un broker MQTT con su correspondiente dirección IP y autenticarse con las credenciales MQTT para ver los tópicos y sus mensajes obtenidos sin la necesidad de construir una aplicación extra para ordenar y mostrar los datos. De esta manera, se corroboró la información enviada por los sensores y se verifica el correcto funcionamiento del protocolo MQTT para este primer prototipo.

En la figura 4.3 se muestra una captura de pantalla de la aplicación MQTT Dash, en el apartado de configuración para conectarse al servidor MQTT, con el correspondiente puerto asignado y las credenciales que identifican al usuario que publica o se suscribe a los tópicos. En este caso, sólo se visualiza un valor de prueba para el sensor de temperatura, ya que los demás se comportan de manera similar.

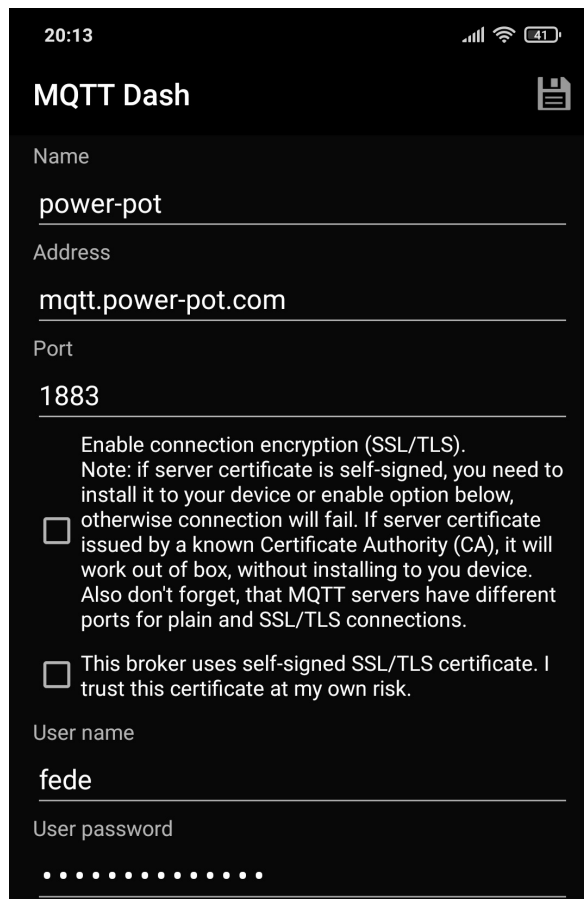


Figura 4.3: Captura de Pantalla de la Configuración de MQTT Dash

Se muestra otra captura de pantalla de la aplicación MQTT Dash en la figura 4.4 , donde se ve un valor arbitrario enviado por el sensor de temperatura al tópico correspondiente.

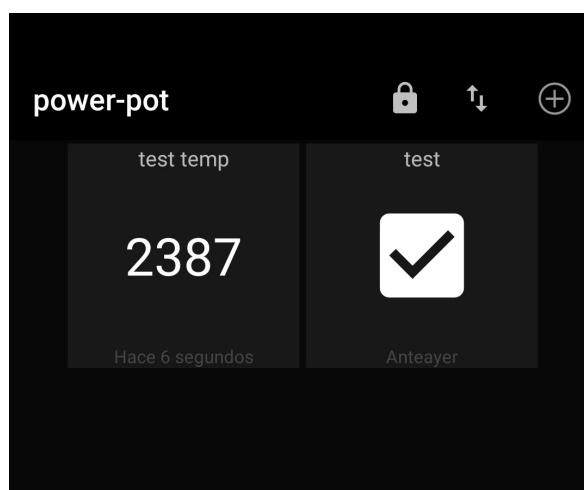


Figura 4.4: Captura de Pantalla de las métricas de MQTT Dash

4.3. Prototipo 2: Flujo de Trabajo

4.3.1. Workflow o Flujo de Trabajo

La primera práctica continua añadida al proyecto fue la **integración continua (CI)**, implementada mediante un **Workflow de GitHub Actions** (véase la sección [Workflows en GitHub Actions](#)), que permite la ejecución de acciones ante determinados eventos. En concreto, **se desea compilar y cargar el código a la placa cada vez que se integre código a la rama principal del repositorio de GitHub**.

El primer paso en la implementación del Workflow fue lograr que el mismo compile el nuevo código una vez que se sube al repositorio, sin la necesidad de subirlo a la placa. Dado que el equipo de trabajo utilizó PlatformIO para compilar, se requirió instalar esta herramienta, bastó con instalar la versión **PlatformIO Core**, es decir, la línea de comandos. De esta manera en lugar de presionar el botón en la interfaz de Visual Studio Code se ejecutó el comando **”platformio run”**, que cumple la misma función de compilar el código pero de forma automatizada gracias a la ejecución del flujo de trabajo en la nube. Para compilar el nuevo binario, PlatformIO descarga e instala las librerías y dependencias que requiere el proyecto, por lo que este proceso suele tomar entre 3 a 5 minutos cada vez que se sube una nueva versión al repositorio de GitHub.

El código del workflow se escribió en **lenguaje YAML**, el cual es un lenguaje de sintaxis sencilla que tiene por objetivo resultar legible para los humanos, y muchas de las tecnologías DevOps admiten archivos en este lenguaje. Este archivo de código se divide en 3 partes fundamentales, los **eventos**, las **variables de entorno**, y los **jobs**. Cada una de estas partes se describe a continuación.

- **Eventos:** Se indica detalladamente ante cuáles eventos debe accionarse el workflow. Los eventos más habituales que desencadenan la ejecución suelen ser **push** (subir nuevo código al repositorio), **pull request** (combinar una rama con otra) y **workflow dispatch** (accionar la ejecución manualmente desde la web de GitHub). Este espacio se identifica con la palabra `.on`.
- **Variables de Entorno:** Se indican cuáles son y qué valores toman las variables de entorno que son utilizadas en las posteriores tareas a ejecutar. Este espacio se identifica con la palabra `.env`.
- **Jobs:** Los jobs son conjuntos de tareas a realizar. Cada Job se identifica a través de un nombre y se indica dónde se debe ejecutar, es decir, si debe ejecutarse en la nube o en un servidor local (self-hosted). Los Jobs deben estar pensados para ejecutarse en simultáneo, es decir, si se construye más de un job, estos podrían ejecutarse de manera concurrente en el mismo servidor, motivo por el cual las

tareas llevadas a cabo por uno no deben depender del resultado de las del otro. A su vez, **cada Job se divide en Steps** (pasos), también identificados por nombres, donde denotan uno o varios comandos a ejecutar. Cabe destacar que si un Job se divide en 3 pasos para pasar al paso 2 debe haberse ejecutado y finalizado con éxito el paso 1, por lo tanto, los pasos sí son dependientes de que los pasos previos hayan sido completados exitosamente.

De esta manera se definió una estructura para las tareas que se deben llevar a cabo y se construyó el flujo de trabajo, que es precisamente este orden en el que deben realizarse las tareas relacionadas con las prácticas continuas.

A continuación se presenta el código del workflow para compilar el binario en la nube.

```
1  name: Build
2  on:
3    pull_request:
4      branches: [ "main" ]
5      paths:
6        - 'src/*'
7        - 'include/*'
8        - 'test/*'
9
10  env:
11    # Direccion del servidor MQTT a conectarse
12    MQTT_SERV: ${ secrets.MQTT_SERV }
13
14  jobs:
15    build:
16      name: "Build"
17      runs-on: ubuntu-latest
18      steps:
19        - uses: actions/checkout@v3
20        - name: "build firmware"
21          run: |
22            pio run --environment nodemcu-32s
```

Extracto de código 4.1: Código Workflow Simple

En el anterior extracto de código se pueden apreciar las 3 partes fundamentales que se mencionaron junto con las líneas de código necesarias para la compilación del programa.

Cabe destacar que en esta instancia no se cargó el binario resultante en la placa

física, ya que como el workflow ejecuta comandos que corren en un servidor en la nube proporcionado por GitHub, por lo tanto, no se tiene acceso físico a los servidores. Esto significa que no es posible conectar un dispositivo físico para la carga del programa, lo cual representa una limitación para el proyecto y se debe buscar una solución. Esta solución es ejecutar el flujo de trabajo de manera local en un Runner Self-Hosted, que se implementa más adelante.

4.3.2. CodeQL

El primer analizador de código que se introdujo al proyecto fue CodeQL, el cual al ser desarrollado por GitHub tiene integración directa con los repositorios alojados allí.

En concreto, se creó un workflow independiente del principal, para ejecutar las tareas correspondientes al análisis del código encontrado en el repositorio de GitHub cada vez que se decida combinar (o mergear) código a la rama principal. Además, también se analizó el código compilado, para buscar vulnerabilidades no sólo en el código escrito por el equipo de trabajo, sino también el código que se añade al incluir librerías externas.

En la figura 4.5 se tiene una captura del escaneo realizado con CodeQL, donde se observa que se encontraron 3 vulnerabilidades del tipo “Comparison of narrow type with wide type in loop condition” (véase la sección [Vulnerabilidades y Amenazas de Seguridad](#)).

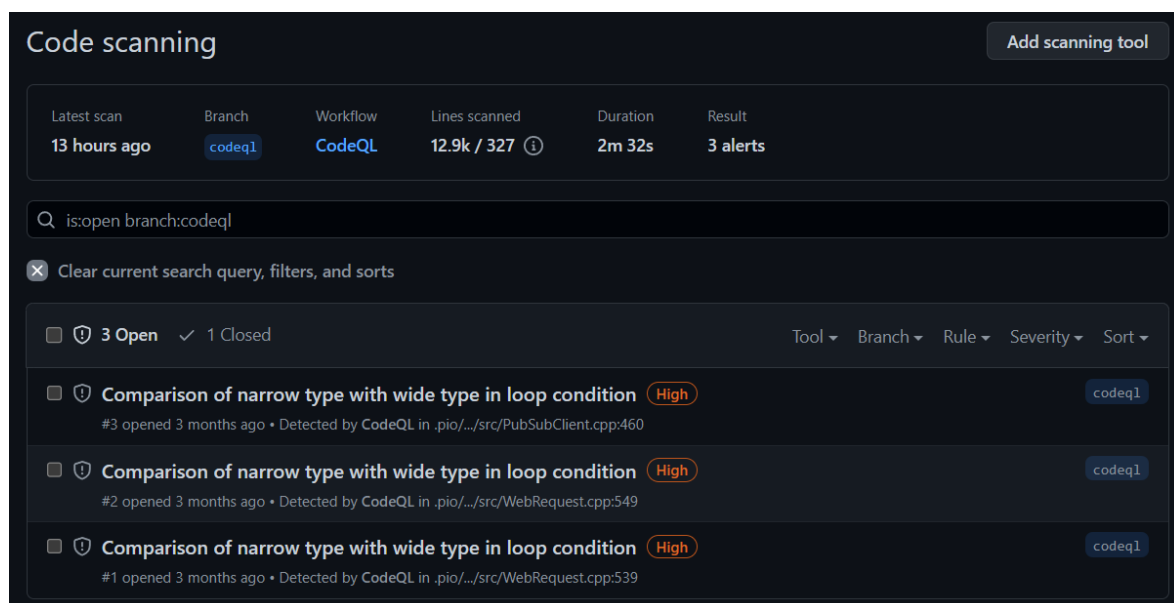


Figura 4.5: Captura de Pantalla del escaneo con CodeQL

Una vez halladas las vulnerabilidades, fue necesario actuar para darles solución de manera temprana en el desarrollo, antes de que el código salga a producción. Por este motivo, fue necesario verificar cuáles de los archivos contienen líneas con vulnerabilidades. En este caso particular, la herramienta ha detectado vulnerabilidades en:

- **/src/PubSubClient.cpp:460** (línea 460)
- **/src/WebRequest.cpp:549** (línea 549)
- **/src/WebRequest.cpp:539** (línea 539)

Se trata de archivos correspondientes a librerías incluidas en tiempo de compilación. Por lo tanto no fue el código que escribió el equipo de trabajo sino código importado desde los repositorios oficiales de estas librerías.

Para subsanar estas vulnerabilidades, dado que se trata de código encontrado en librerías de código abierto, se realizó un fork (o bifurcación) de los repositorios oficiales de las librerías **PubSubClient**[33] y **ESPAsnycWebServer**[39], respectivamente. Las modificaciones efectuadas para solucionar los problemas encontrados por la herramienta CodeQL se corrigieron siguiendo las recomendaciones encontradas en los enlaces brindados por la misma. Como se trata de 3 vulnerabilidades del mismo tipo, la solución consiste en verificar cuales son las condiciones de los bucles que dan origen a la debilidad. Al haber cambiado los tipos de datos de las variables a comparar para que coincidan en ancho de bits, la potencial vulnerabilidad de quedar en un bucle infinito desaparece, siendo comprobado por un posterior análisis de CodeQL, en el cual ya no aparecieron las 3 alertas de alta prioridad mostradas en la figura 4.5.

El paso siguiente a haber subsanado las 3 debilidades en el código de las librerías fue indicar en la configuración del proyecto, en concreto en el archivo platformio.ini ubicado en el directorio raíz del repositorio, que se debe hacer uso de los repositorios fork [12] [11] (o bifurcaciones) en lugar de los repositorios oficiales.

4.3.3. NodeRED

NodeRED es una herramienta de programación visual de código abierto que se utiliza para conectar dispositivos y servicios en la nube de una manera fácil y rápida, por lo que resulta ideal para aplicarlo a proyectos Internet of Things (IoT).

NodeRED proporciona una interfaz gráfica en el navegador web que permite crear flujos de trabajo arrastrando y soltando nodos en un lienzo. Estos nodos representan diversas funcionalidades, como entradas, salidas, procesamiento de datos y lógica. Se pueden conectar estos nodos para crear flujos que manejan tareas como recopilación de

datos, transformación, filtrado, almacenamiento y envío de información entre diferentes sistemas y dispositivos.

La incorporación de NodeRED a este proyecto resultó en un avance significativo porque permitió implementar una interfaz web en la que se muestran los datos recibidos por los sensores de una manera clara y ordenada para el usuario, en lugar de usar la aplicación de celular que resulta algo más incómoda y poco clara, ya que la misma sólo se usó con motivos de prueba.

En la figura 4.6 se observa el tablero o dashboard web en el que se muestran las métricas de los sensores del prototipo en tiempo real.

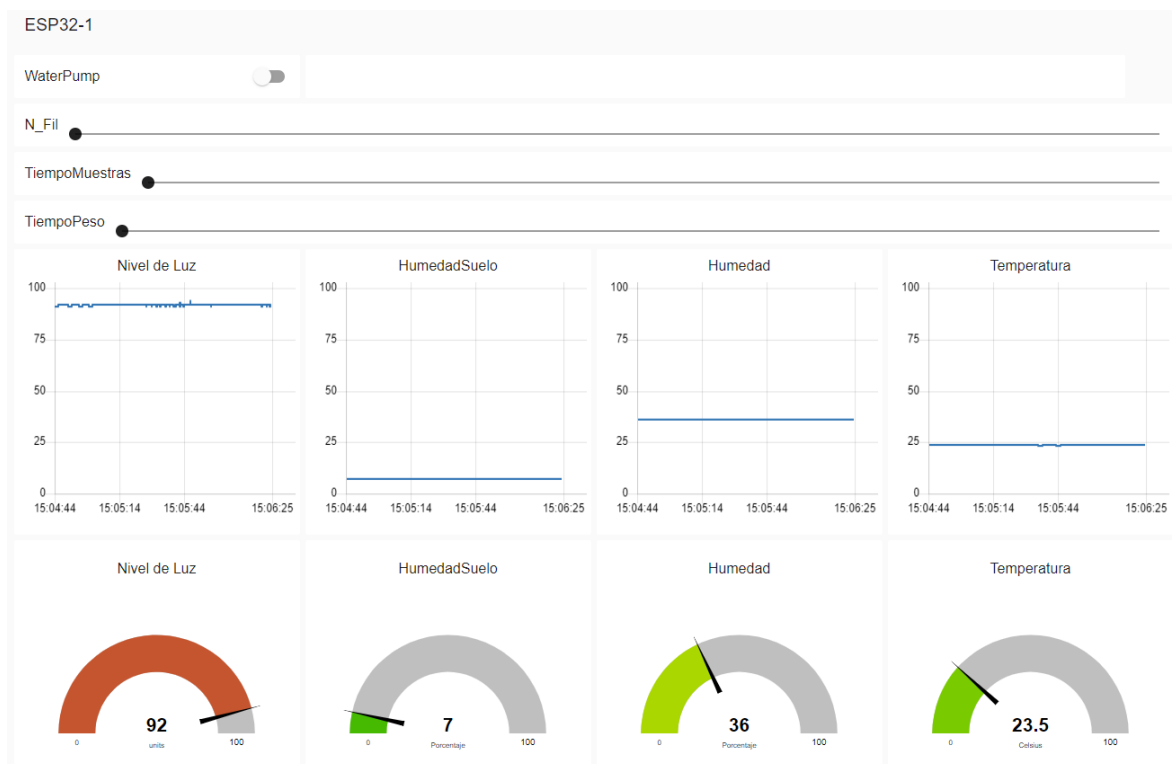


Figura 4.6: Dashboard de Métricas de los Sensores

La figura 4.7 muestra el entorno de NodeRED donde se ve el flujo de nodos visuales y la interconexión de los mismos para manipular la información de los tópicos MQTT.

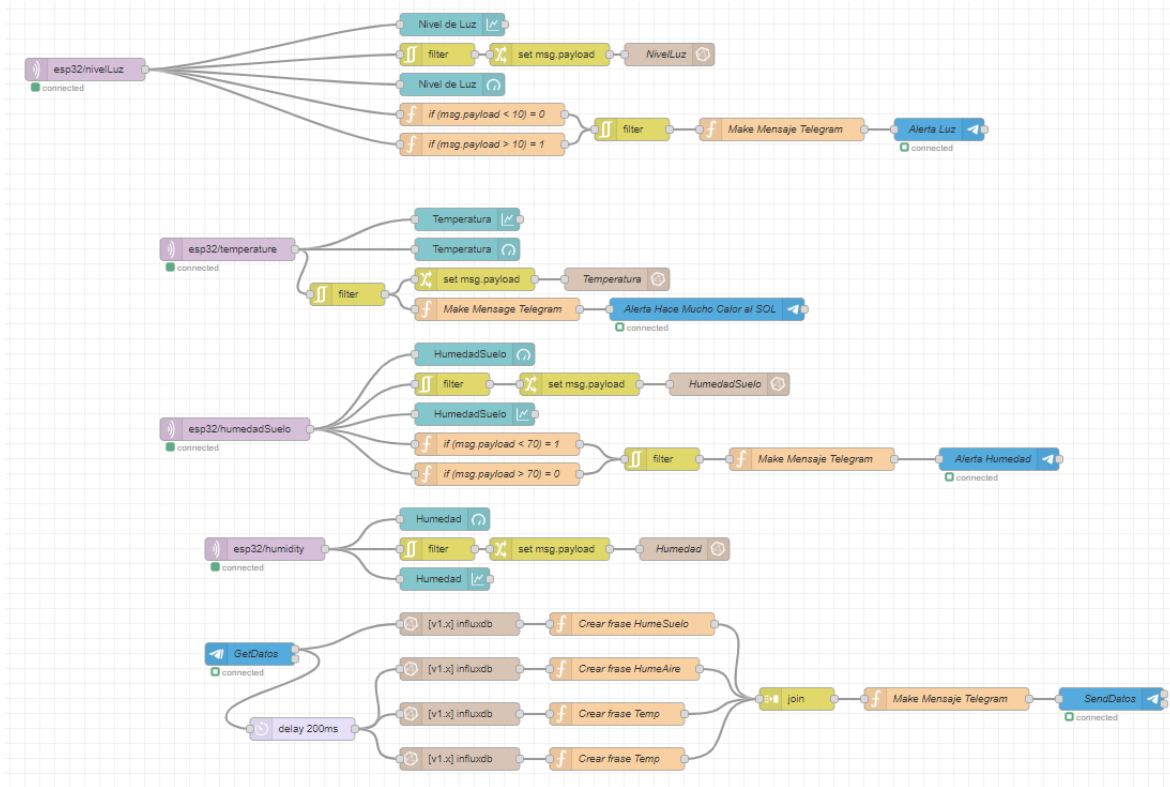


Figura 4.7: NodeRED: Flujo de Nodos

4.4. Prototipo 3: Ecosistema de Servicios en Raspberry Pi

4.4.1. Runner Self-Hosted

En este paso se requirió implementar un **Runner Self-Hosted**, descrito en detalle en la sección [Runners](#), el cual es una instancia de GitHub Actions que corre de manera local en una máquina a la que se tiene acceso, permitiendo la posibilidad de conectarle la placa ESP32 y tener acceso a ella para cargar el código nuevo que incluye las modificaciones de la última versión encontrada en el repositorio de GitHub.

Para hacerlo se siguió la documentación oficial de GitHub para añadir runners a un repositorio [27]. De forma resumida, el procedimiento a llevar a cabo consistió en descargar el último lanzamiento (o release) del Runner oficial [28], en este caso la versión específica para Linux en arquitectura ARM (correspondiente a la Raspberry Pi), descomprimir y configurar el runner indicando la dirección web del repositorio de GitHub al cual se lo desea añadir, y además brindarle un token, que es un código de

acceso alfanumérico que debe ser generado por el dueño del repositorio al que se quiere añadir el runner.

Una vez descargado y configurado el runner, se inició el proceso de autenticación con los datos previamente configurados para conectarse a la API de GitHub y permitir que se reciban las tareas del Workflow y se puedan ejecutar utilizando la potencia computacional de la máquina local.

La figura 4.8 muestra una captura de pantalla de la terminal de Linux ejecutando los comandos de configuración y ejecución del runner self-hosted hasta quedar en el estado "Listening for Jobs", es decir, a la espera de Jobs para ejecutarse de forma local. Cada vez que un job se ejecute se indica en esta terminal con el detalle de la fecha, hora y nombre del job que se ejecuta. Una vez finalizada la ejecución, la terminal muestra si el resultado del job fue exitoso (Succeded) o si tuvo alguna falla (Failed), en cuyo caso se debe proceder a revisar los logs para visualizar en qué comando del workflow se obtuvo el fallo y cuál fue el error.

```

      _____
     /          \
    /             \
   /               \
  /                 \
 /                   \
/                     \
\                     /
 \                   /
  \                 /
   \               /
    \             /
     \          /
      \       /
       \    /
        \_/

Self-hosted runner registration


# Authentication

✓ Connected to GitHub


# Runner Registration

Enter the name of runner: [press Enter for linux-runner]

✓ Runner successfully added
✓ Runner connection is good


# Runner settings

Enter name of work folder: [press Enter for _work]

✓ Settings Saved.

linuxadmin@linux-runner:~/actions-runner$ ./run.sh

✓ Connected to GitHub

2023-11-06 03:22:25Z: Listening for Jobs
```

Figura 4.8: Captura de Pantalla del Runner Self-Hosted

Gracias a la adición de esta característica, este prototipo quedó con la capacidad de actualizar una placa ESP32 con la nueva versión del código a través de cable USB cada vez que se agrega código nuevo a la rama main del repositorio de GitHub. Esto se consiguió agregando un paso adicional al job que corresponde a la compilación del código.

El proceso de combinar una rama para integrar su código a la rama principal puede ser realizado desde cualquier computadora con internet estando en cualquier parte, ya que este proceso sólo implica subir la nueva versión a GitHub. Una vez que el workflow detecta que se tiene una nueva versión, comienza la ejecución de las tareas para compilar y subir el nuevo programa a la placa que debe estar conectada físicamente por cable USB al servidor en el que corre el runner, en este caso al servidor Raspberry Pi.

Este proceso de actualización demora aproximadamente 6 minutos la primera vez, y unos 3 minutos las siguientes veces. Esto se debe a que **una parte importante del tiempo inicial es el que se requiere para descargar e instalar la herramienta PlatformIO y sus dependencias** al runner, mientras que las siguientes veces esas dependencias ya se encuentran instaladas y sólo se necesita compilar y cargar el programa en la placa, siempre y cuando no se haya cerrado el proceso del runner.

Además este prototipo tuvo otro inconveniente, y es que **no corría el runner en cualquier distribución de Linux**, ya que algunas no tienen instaladas los requerimientos para que funcione la aplicación del runner. Por ejemplo, en Ubuntu y Ubuntu Server el runner corre sin problemas ni necesidad de instalar componentes adicionales, mientras que en LinuxMint (otra distribución popular de Linux basada en Ubuntu) no corría y fue necesario investigar qué paquetes fueron necesarios para ejecutarlo.

4.4.2. Servidor Web Nginx

Para servir un sitio web se usó **Nginx**, el cual es un servidor web liviano y de código abierto que permite definir una o varias interfaces web que se muestran al acceder a una dirección a través de un navegador.

Nginx utiliza el protocolo HTTP (véase la sección [Protocolo HTTP y HTTPS](#)), el cual por defecto usa el puerto 80. Para acceder al sitio web desde una red externa es necesario **redirigir el puerto (Port Forwarding)**. Esta práctica consiste en indicarle al Router de la red que cuando una petición solicita el servicio asignado al puerto 80, debe ser redireccionado al IP privado correspondiente al servidor, en este caso, al servidor Raspberry.

4.4.3. Servicios de Monitoreo

Se añadieron además servicios que permiten el monitoreo continuo del hardware, para visualizar métricas tales como el porcentaje de uso del CPU y memoria, temperatura del dispositivo, entre otras. Para lograr esto se incorporaron varios servicios que se describen a continuación.

- **Prometheus:** Es una solución integral de monitoreo enfocada en la recopilación, almacenamiento y análisis de métricas. Opera bajo un modelo de recopilación pull, en el cual los componentes de monitoreo exponen métricas específicas. De esta manera, Prometheus realiza extracciones periódicas y sistemáticas de dichas métricas.
- **Node Exporter:** Recopilación de métricas del sistema operativo y del hardware del sistema anfitrión. Estas métricas abarcan una variedad de aspectos, como el uso de la CPU, la memoria, el almacenamiento, la red y otros recursos del sistema.
- **cAdvisor:** Es un componente para monitorear contenedores de Docker, tiene la responsabilidad de recolectar métricas específicas de los contenedores en tiempo real. Además de brindar información detallada sobre la utilización de recursos por cada uno de los contenedores, incluyendo aspectos como la asignación de CPU, la utilización de memoria, el uso del disco y la actividad de red.
- **InfluxDB:** Se ha adoptado InfluxDB como una solución de base de datos. Se emplea para asegurar la integridad y disponibilidad de las métricas originadas en los sensores de los dispositivos ESP32. Esta herramienta permite una consolidación eficiente de las mediciones, brindando redundancia y respaldo a los datos esenciales.
- **Grafana:** Esta herramienta se empleó como solución para llevar a cabo la extracción y análisis de datos, además de presentarlos en esquemas personalizados que permiten seleccionar cuáles de todas las métricas recopiladas deben mostrarse y en qué disposición para que resulte claro y ordenado según la necesidad.

En la figura 4.9 se ilustra un diagrama de arquitectura de los servicios antes mencionados y su comunicación para lograr presentar el tablero o dashboard de monitoreo del hardware.

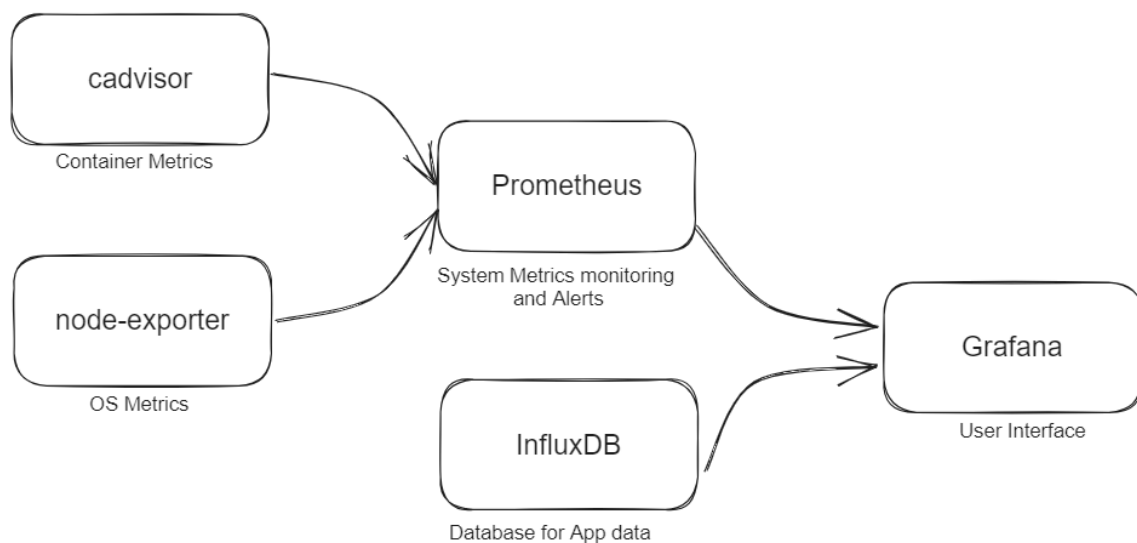


Figura 4.9: Arquitectura de los Servicios de Monitoreo

En la figura 4.10 se observa una captura de pantalla del tablero de Grafana donde se visualizan las métricas correspondientes al estado del hardware en tiempo real. De derecha a izquierda se tiene el tiempo desde que se inició el servidor, la cantidad de contenedores ejecutándose, el porcentaje de uso del CPU y de memoria RAM, promedio de carga del sistema, uso del disco y temperatura del procesador.



Figura 4.10: Captura de Pantalla del Tablero de Grafana

4.4.4. Nginx Proxy Manager

Otra herramienta que facilitó la gestión es Nginx Proxy Manager, que consiste en un servidor proxy inverso (véase la sección [Proxy](#)). Se utiliza para redirigir y administrar el tráfico web entrante de manera eficiente entre diferentes servicios, además de permitir la configuración de certificados SSL para establecer una comunicación segura.

A continuación se presenta la figura 4.11, en la que se puede observar un diagrama que muestra el objetivo del servidor proxy inverso, que redirige el tráfico web entrante al servidor en función de cuál es el subdominio con el que se accede, lo que determina a cuál de los servicios disponibles se desea conectar.

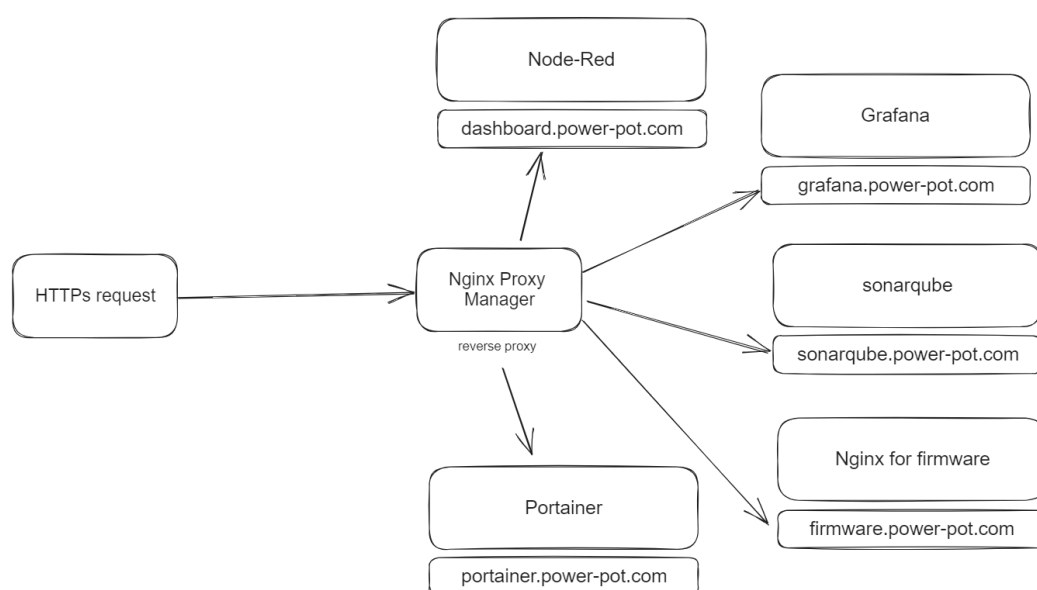


Figura 4.11: Flujo de Subdominios

4.4.5. Portainer

Portainer es una herramienta de código abierto que se usa para la gestión de contenedores, lo que resulta en una facilidad a la hora de administrar contenedores de Docker, en especial en entornos donde estos son numerosos. Su interfaz gráfica intuitiva permite visualizar, gestionar y configurar tanto contenedores, imágenes, redes y volúmenes en el servidor.

El equipo de trabajo utiliza esta herramienta para ver cuando un contenedor colapsó y se detuvo, leer detenidamente sus logs y identificar el origen del problema, para luego reiniciar el contenedor, desde la misma interfaz. Si bien este proceso puede

realizarse a través de comandos en la terminal, es más fácil y rápido identificar y resolver problemas en los contenedores con esta herramienta.

La figura 4.12 muestra una captura de pantalla del tablero de control de Portainer, en la sección de contenedores, donde se ven contenedores corriendo de los diversos servicios que se ofrecen.

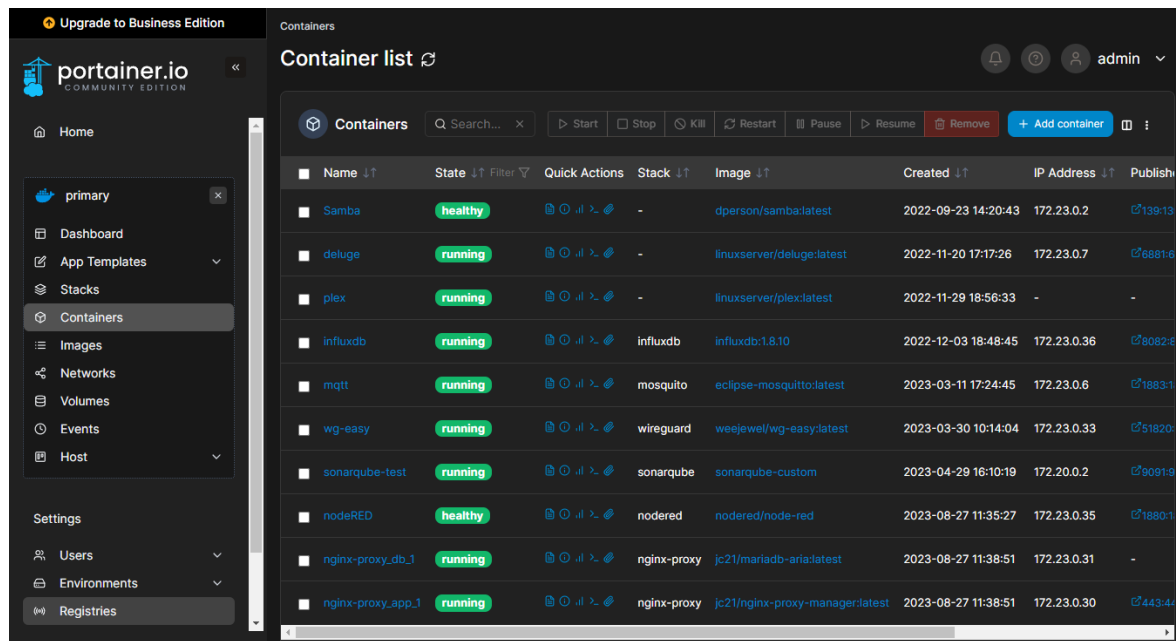


Figura 4.12: Captura de Pantalla de Portainer

4.5. Prototipo 4: Docker y Testing

4.5.1. Imagen de Docker

Para solucionar los inconvenientes del prototipo anterior se presentó una solución basada en virtualizar un entorno, en el cual se instalaron los paquetes necesarios de antemano, permitiendo que las aplicaciones corran en cualquier sistema operativo. Consiste en crear una **imagen de Docker** (véase la sección [Docker](#)) en la cual se incluyó, además del ejecutable del runner, las dependencias que este necesita para funcionar, y además la herramienta **PlatformIO Core**, que consiste solamente en la línea de comandos de PlatformIO. De esta manera se evitó descargar e instalar esta herramienta cada vez que se cierra y se abre el runner.

La creación de la imagen de Docker se realizó a través de un archivo llamado **Dockerfile**, en el cual se describen las características que debe tener la imagen. Por lo

general se parte de otra imagen, en este caso se partió de la imagen de Ubuntu 20.04, para acceder al gestor de paquetes apt que se usa para instalar las mismas dependencias que se hubieran instalado en el servidor Raspberry donde corría previamente el runner. Se indicaron en el archivo cuales son los paquetes de dependencias que se desean instalar, siendo las principales Python y PlatformIO. En la imagen se añadió el enlace desde el cual se debe descargar el runner, y se lo descomprime.

De esta manera se construyó la imagen que servirá como base para instanciar un **contenedor de Docker**, que es el entorno virtualizado necesario para solucionar los inconvenientes del anterior prototipo. La imagen es como una receta que permite instanciar uno o varios contenedores con la misma aplicación. Para este propósito, no es necesario instanciar más de un runner, pero si se buscara sumar la potencia de un segundo servidor para ejecutar tareas del workflow, bastaría con instanciar otro contenedor a partir de la misma imagen creada anteriormente.

Para llevar a cabo este proceso de "dockerizar" el runner, se tomó como base el proyecto de código abierto de Thomas Cardonne[8]. Sólo se basó en la idea de construir un **entrypoint o punto de entrada**, que consiste en un script a través del cual se solicita a la API de GitHub la información para configurar el runner antes de ejecutarlo. La imagen en sí, se construyó desde cero ya que las dependencias son principalmente las necesarias para ejecutar PlatformIO, la cual no está contemplada en el proyecto de Cardonne.

Es importante tener en cuenta que el proyecto utiliza librerías personalizadas (o custom), que provienen de un repositorio personal, donde se han subsanado las vulnerabilidades encontradas por los analizadores. Para incluir esta característica, simplemente se debe instalar git en el contenedor correspondiente al runner self-hosted. De esta manera, PlatformIO puede ejecutar un **git pull** y hacerse de las librerías personalizadas.

4.5.2. Docker Compose

Además del archivo que describe la imagen, se trabajó en implementar un archivo de **Docker Compose**, que consiste en un archivo que detalla cuántos contenedores instanciar, a partir de qué imagen, como también cuáles son las variables de entorno que se deben considerar.

En este caso particular resultó importante pasar como variables de entorno tanto el enlace del repositorio al que se debe asociar el runner, como también el token de acceso. Sin embargo, ya que el archivo de Docker Compose queda escrito y se sube al repositorio de código abierto, no deben incluirse los valores de estas variables sino solamente sus nombres, para no comprometer la seguridad del proyecto. Los valores

de estas variables se incluyen en un archivo separado, con extensión ".env" diseñado específicamente para esta finalidad, de modo que se puede subir al repositorio los archivos de Docker exceptuando aquellos con extensión ".env", indicando esta excepción en el archivo .gitignore.

Dentro del Docker Compose, se indicaron también **Volúmenes y Dispositivos**, en los apartados volumes y devices, respectivamente. En el primero se mapearon carpetas del sistema anfitrión con carpetas dentro del contenedor de docker o volúmenes, siendo el sistema anfitrión el servidor Raspberry y el contenedor es el entorno donde corre el runner. En el apartado devices se mapearon los puertos, siendo necesario mapear el puerto USB0 de la Raspberry al puerto USB0 del contenedor, de esta manera se lo tiene accesible para escribir a través de éste cuando sea el momento de cargar el binario a la placa ESP32.

Esta práctica de desplegar servicios con archivos Docker Compose se realizó también con el resto de los servicios mencionados en el prototipo 3, cada uno con sus respectivos Compose y archivos de configuración.

4.5.3. DockerHub

Una vez creada la imagen de Docker, y luego de haberla probado en un entorno local, se procedió a asignarle un nombre y una etiqueta (o tag), para posteriormente subirla a DockerHub.

Esta plataforma consiste en un registro que permite crear repositorios para alojar imágenes de Docker. Se podría comparar con GitHub pero la diferencia sustancial es que no se sube el archivo Dockerfile, que determina las características que debe tener la imagen, sino que se sube la imagen construida, junto con todas sus dependencias ya empaquetadas. Es importante notar esta diferencia ya que el archivo Dockerfile consta simplemente de un archivo de texto, y sólo pesa unos pocos kilobytes, mientras que la imagen ya construida suele pesar varios megabytes. Además el proceso de construir la imagen a partir del Dockerfile suele durar varios minutos y requiere conexión a internet.

La ventaja que se obtiene de subir la imagen completa a DockerHub es permitirle a los desarrolladores ejecutar contenedores a partir de la imagen en cualquier sistema en cuestión de pocos minutos. Si en el futuro se decidiera cambiar el servidor Raspberry por uno más potente, se podría construir exactamente el mismo entorno contenerizado donde se encuentra el runner, haciendo uso de esta imagen alojada en la nube y del archivo Docker Compose antes mencionado. Dado que el Compose sólo indica el nombre de la imagen a instanciar, en el caso que ésta no se encuentre en el sistema anfitrión, entonces se la buscará en DockerHub.

En este caso particular la imagen creada por el equipo de trabajo lleva el nombre de **feedehc/github-docker-runner:linux-arm**, donde **feedehc** es el nombre de usuario de la plataforma DockerHub, **github-docker-runner** es el nombre de la imagen y **linux-arm** es la etiqueta o tag asociada a la imagen, ya que una imagen podría tener variantes para diferentes arquitecturas de procesador como en este caso, que se aclara que es para arquitectura ARM. La imagen lista pesa alrededor de 300 MB, incluyendo todo lo necesario para funcionar en cualquier sistema operativo de arquitectura ARM.

4.5.4. Tests Automatizados

El próximo paso que se implementó fue añadir tests automatizados como un paso (o step) extra en el flujo de trabajo principal. La idea es que sólo debe permitirse realizar la compilación y distribución de una nueva versión cuando ésta haya sido ejecutada y probada con éxito en una placa ESP32 de pruebas. La placa de pruebas debe estar conectada todo el tiempo al servidor Raspberry, de modo que los tests puedan correrse sin intervención de un operario.

Los tests son simplemente un binario especial que implementa el **framework de testing Unity de ThrowTheSwitch**. Este framework permite crear pruebas de tipo Assert, que son simplemente pruebas de verdad, en las que se compara el resultado de una función con un valor esperado. Por ejemplo, para probar la conexión al broker MQTT, se realiza un **TEST_ASSERT_EQUAL()** en el cual se compara el valor **true** con el resultado de la función **pubSubClient.connected()** ya que se espera que la función devuelva el valor verdadero luego de haber establecido la conexión exitosa.

Es importante destacar que dado que en la estructura del proyecto las funciones se escriben en un archivo de código aparte del principal, llamado **functions.cpp**, se realizan las pruebas con las mismas funciones que se implementan en el programa principal. El archivo **test_main.cpp** actúa como un main secundario que sólo compara resultados de la ejecución de funciones reales con valores esperados, y si todas las pruebas se completan con éxito, entonces el resultado de las pruebas es positivo y se puede continuar con los siguientes pasos del workflow de GitHub Actions.

4.6. Prototipo 5: Actualizar por WiFi

4.6.1. Actualizaciones por WiFi

El primer paso que se llevó a cabo para implementar las actualizaciones inalámbricas fue utilizar la **librería AsyncElegantOTA** la cual soporta integración con el

framework Arduino.

En pocas palabras, esta librería ofrece la posibilidad de que la placa ESP32 actúe como servidor web, permitiendo la conexión al mismo a través de un navegador web, utilizando la dirección IP privada asignada a la ESP32, cuando ésta se conecta a la red WiFi. Una vez que se accede por el navegador se tiene la posibilidad de cargar un binario para que se actualice, y luego de reiniciarse, el dispositivo iniciará con su nueva versión del programa.

En la figura 4.13 se observa una captura de pantalla del navegador web en la interfaz donde se añade el firmware de forma manual para actualizar la placa ESP32.

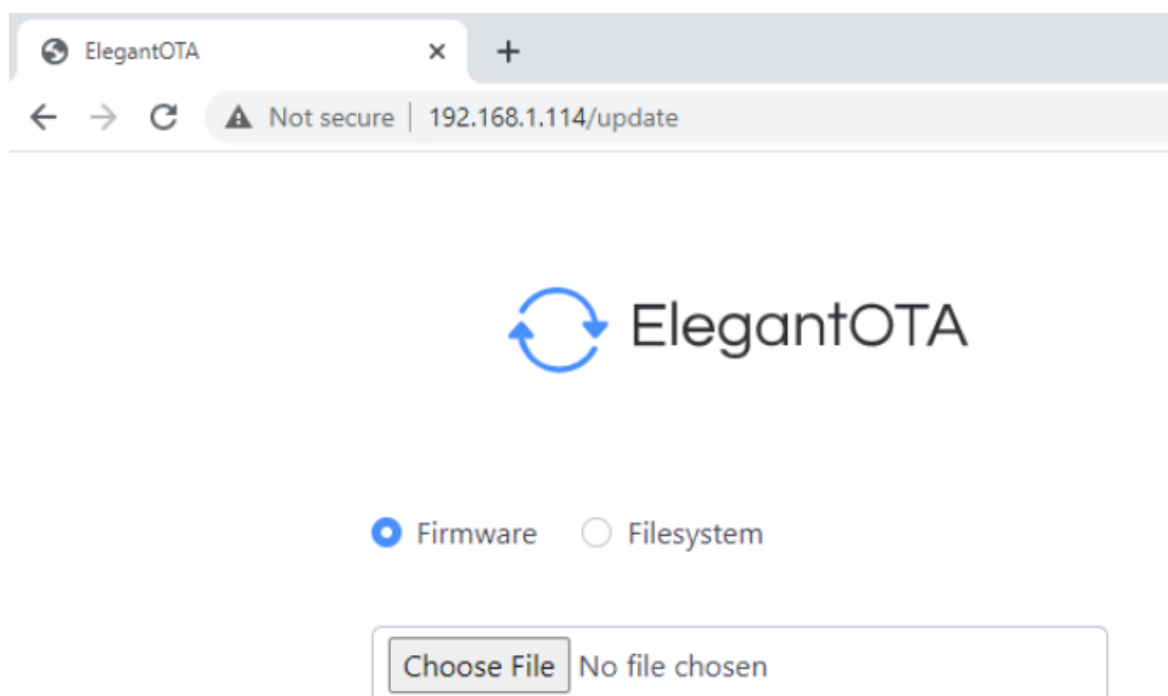


Figura 4.13: Captura de Pantalla de ElegantOTA

El inconveniente que tiene esta metodología es que **el binario debe cargarse manualmente**, no se dispone un comando con el cual se pueda automatizar la tarea, motivo por el cual se vuelve necesario buscar una alternativa con esta característica.

La alternativa que se implementó para realizar la carga de manera automática, fue elaborar un script en lenguaje Python llamado **platformio_upload.py** con el objetivo de enviar por WiFi el binario hacia la placa ESP32 cuando se utiliza el comando **platformio run**, aclarando en las configuraciones que se debe usar un método personalizado (upload custom), que consiste en ejecutar el script anteriormente mencionado.

Esta primera forma de actualizar los dispositivos por aire implicó una gran ventaja y comodidad a la hora de desarrollar, sin embargo, resultó poco escalable, ya que para enviar el binario por WiFi se necesita la dirección IP del dispositivo que debe recibir la actualización. En el caso que se tuvieran varias placas, este proceso puede resultar incómodo, ya que se debe conocer cuántas placas hay y sus respectivas direcciones IP.

4.6.2. Cppcheck

Luego se incluyó al proyecto el analizador **Cppcheck**. Este analizador está soportado oficialmente por la herramienta PlatformIO, por lo tanto resulta sencillo añadir la posibilidad de analizar el código de manera local con Cppcheck.

Para añadir el analizador local al proyecto se siguieron los pasos de la documentación oficial[44], que consisten básicamente en añadir en el archivo de configuración "platformio.ini" la línea `check_tool = cppcheck` para indicar que esta será la herramienta a ejecutar cuando se ejecute el comando "**platformio check**". De esta manera, cada vez que se ejecute el comando mencionado, se obtendrá un análisis estático del código fuente del proyecto.

El propósito de esta herramienta es incluirla al workflow general en un paso (o step) adicional, antes de compilar el código. Además sirve para brindar a los desarrolladores una manera de analizar el código que van agregando al proyecto antes siquiera de subirlo al repositorio. Al estar integrado con PlatformIO se puede ejecutar fácilmente con el comando mencionado o con el botón check en la interfaz gráfica.

En la figura 4.14 se observa el resultado del análisis automático ejecutado como parte del flujo de trabajo principal, en un paso previo a la compilación del programa.

platformio check with cppcheck						
Component		HIGH	MEDIUM	LOW		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Array		0	2	4		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Collection		0	0	2		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Deserialization		0	0	4		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Deserialization/Readers		0	6	0		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Document		0	8	14		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Json		0	4	12		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Memory		0	0	4		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/MsgPack		0	0	12		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Numbers		0	0	6		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Object		0	2	6		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Polyfills		0	0	2		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Serialization/Writers		0	4	0		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/StringStorage		0	8	4		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Strings		0	0	2		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Strings/Adapters		0	0	6		
.pio/libdeps/nodemcu-32s/ArduinoJson/src/ArduinoJson/Variant		0	4	12		
.pio/libdeps/nodemcu-32s/PubSubClient/src		0	0	2		
.pio/libdeps/nodemcu-32s/WiFiManager		0	0	4		
nofile		0	0	2		
src		0	0	2		
Total		0	38	100		
Environment	Tool	Status	Duration			
nodemcu-32s	cppcheck	PASSED	00:00:06.761			
===== 1 succeeded in 00:00:06.761 =====						
Platformio Check finalizado						

Figura 4.14: Captura de Pantalla del escaneo con Cppcheck

4.6.3. SonarQube

Por último se añadió el analizador SonarQube. Esta herramienta ofrecida por SonarSource tiene como característica principal la posibilidad de ejecutarse en un servidor local, de manera análoga a como se ejecuta el servicio del self-hosted runner en el servidor Raspberry Pi. Esto permitió al equipo de trabajo tener un mayor control sobre la gestión de los recursos.

El método utilizado para instalar esta herramienta es a través de un archivo Docker Compose, del mismo modo que se hizo con los servicios del prototipo anterior. Además se añadieron las configuraciones para vincular el repositorio del proyecto para su posterior análisis.

Es importante mencionar que esta herramienta posee una versión de prueba con limitaciones, en concreto **se limita a analizar archivos de varios lenguajes, como**

por ejemplo Python, pero no analiza archivos en lenguaje C++, que es el principal lenguaje en el cual se escribe el código del proyecto. El equipo de trabajo investigó acerca de un plugin necesario para que la herramienta detecte los archivos C++, y a pesar de que la misma logra detectarlos, no muestra las debilidades porque esa es una característica de la versión de pago. Es por este motivo que SonarQube sólo se añade a modo ilustrativo. **Se incluye al proyecto como oportunidad de analizar código Python para otros desarrolladores**, pero para esta aplicación en particular no resultó en un beneficio sustancial.

En la figura 4.15 se observa el resultado del análisis de código efectuado por SonarQube en los archivos de python del proyecto. En este caso particular, la herramienta sugiere que no se use el protocolo HTTP en el script de Python para subir el binario por WiFi, y reemplazarlo por su variante encriptada, el protocolo HTTPS. Este script se dejó de usar en el siguiente prototipo.

The screenshot displays the SonarQube web interface. At the top, there's a navigation bar with the SonarQube logo and various menu items. Below it, a search bar and a status bar show the last analysis date and warning count. The main area is split into a sidebar and a main pane. The sidebar contains filters and a list of security hotspots. The main pane shows the details of a specific hotspot, including its title, description, status, and the relevant code snippet from the file 'platformio_upload.py'. A red banner highlights the issue, and a 'Comment' button is visible.

Figura 4.15: Captura de Pantalla del las debilidades encontradas por SonarQube

4.7. Prototipo 6: Actualizaciones Automáticas por HTTP

4.7.1. Actualización Automática por HTTP

Para dar solución a los inconvenientes en las actualizaciones inalámbricas del anterior prototipo, se planteó una forma diferente de actualizar los dispositivos. Para lograr una mayor escalabilidad, el modelo de actualización coloca el binario de cada nueva versión del programa en un servidor web. El servidor web utilizado es Nginx.

Este servidor web utiliza el **protocolo HTTP** (véase la sección [Protocolo HTTP y HTTPS](#)) para servir los archivos de la misma manera que se sirven sitios web y archivos para descargar por navegador. En concreto se tienen 2 archivos, por un lado el binario con la última versión disponible, y por otro lado un archivo **JSON (JavaScript Object Notation)** que indica cuál es el número que corresponde a la última versión. De esta manera, se construyó una función para que las placas ESP32 comparen frecuentemente el número que indica el archivo JSON con la versión que tienen instalada en ese momento. Si se detecta que hay una versión posterior disponible, se inicia el proceso de descarga y actualización, para luego reiniciarse y comenzar a usar el nuevo programa.

La función para comprobar las actualizaciones se llama **checkFirmwareUpdate()** y se ejecuta una vez cada cierto tiempo, definido a través de un parámetro. La forma en la que funciona es descargando el archivo JSON antes mencionado, del cuál se extrae el número correspondiente a la última versión disponible online, y luego de compararlo se decide si debe actualizarse o no. Este proceso imprime mensajes en el **monitor serial**, de modo que si se conecta la placa ESP32 a una PC por medio del cable USB se pueden leer los mensajes que indican las versiones que se están comparando, y el momento en que la placa se está reiniciando para aplicar la actualización. Todo el proceso de actualización demora aproximadamente 1 minuto.

4.7.2. Compilación y Disponibilidad del Binario

Una vez lograda la actualización, es importante automatizar el proceso de compilación y puesta en disponibilidad del binario que corresponde a la nueva versión que se pretende actualizar.

Para llevar a cabo este paso se modificó el workflow destinado a compilar y subir el binario, que en el anterior prototipo subía el binario por WiFi haciendo uso del script

de Python. En este caso, se indica al workflow que sólo debe compilar el binario sin subirlo, y cuando se tiene el archivo resultante, llamado "firmware.bin", se agrega el numero de versión al final, por ejemplo "firmware-0.52.bin".

Finalmente, se copia a la carpeta correspondiente al servidor web Nginx. Este proceso se realiza gracias al **mapeo de volúmenes de Docker**, que permite relacionar una carpeta dentro de un contenedor con otra fuera del contenedor. Debe tenerse en cuenta que la compilación se realiza dentro del contenedor de Docker del Runner Self-Hosted.

En la figura 4.16 se observa la vinculación entre los contenedores del servicio Nginx que ofrece el firmware a las placas, y el Runner que lo publica cada vez que termina de compilar una nueva versión que pasa con éxito todas las pruebas. La publicación se hace efectiva gracias a un volumen de Docker que permite la comunicación entre ambos contenedores.

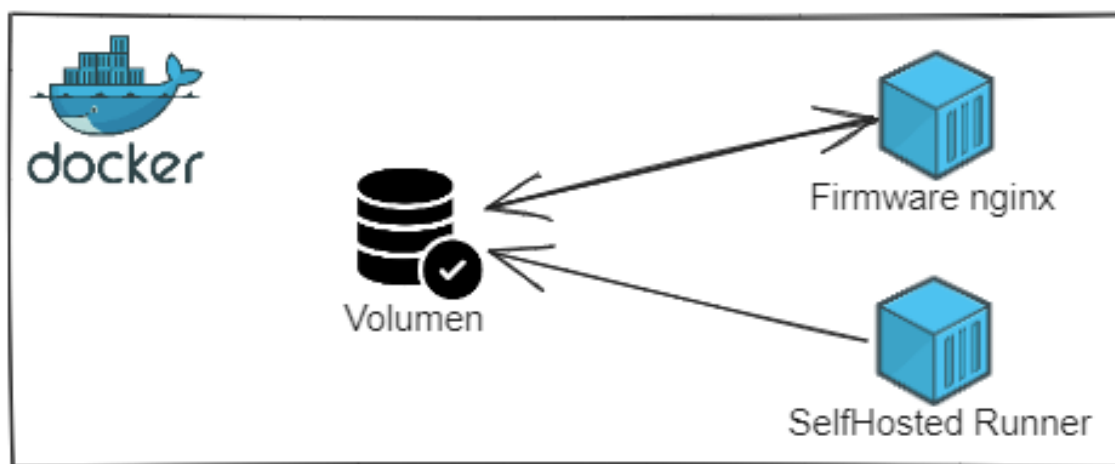


Figura 4.16: Volumen de Docker para el Firmware

4.7.3. Certificado HTTPS y Dominio DNS

Es importante destacar que para mantener la seguridad en el proceso de actualización, se requirió implementar el **protocolo HTTPS** en lugar de HTTP. Para esto se utilizó un **certificado SSL** emitido por una autoridad certificadora (CA por sus siglas en inglés).

Este proceso se implementó contratando un plan anual de GoDaddy, que incluye dominio DNS, certificado SSL, entre otras configuraciones para poner en línea un sitio web.

Luego se configuraron los registros de CloudFlare con los subdominios correspondientes a los servicios que se disponen en el servidor, los cuales se describen a continuación:

- **firmware.power-pot.com:** Redirige al servicio Nginx para descargar la última versión del binario para instalar en las placas ESP32.
- **dashboard.power-pot.com:** Redirige al servicio de NodeRED donde se ven las métricas de los sensores. Además, se puede añadir el sufijo /admin para acceder al panel de configuración del flujo de nodos.
- **grafana.power-pot.com:** Redirige al servicio Grafana donde se ve un tablero con métricas de monitoreo del hardware del servidor y de las placas.
- **portainer.power-pot.com:** Redirige a la interfaz web de Portainer, para visualizar y administrar los contenedores de Docker que corren en el servidor.
- **sonarqube.power-pot.com:** Redirige a la interfaz web de SonarQube, donde se pueden ver las alertas de seguridad y recomendaciones a tomar.

4.7.4. Gestión de Credenciales con WiFiManager

Un aspecto importante que se necesita modificar luego de tener este nuevo método de actualizaciones automáticas, es la gestión de las credenciales de WiFi. Dado que se tiene un único binario disponible para todas las placas ESP32, es necesario ofrecer la posibilidad que el cliente configure sus credenciales de forma local, ya que éstas no deben compartirse por internet, por motivos de seguridad.

Esta característica se implementó gracias a la **librería WiFiManager**, la cual configura la placa ESP32 como Access Point, es decir, que emite una red WiFi con contraseña, para conectarse a ella desde el teléfono celular. Una vez conectado se muestra una página de configuración en la que se elige entre las redes WiFi disponibles dentro del alcance, se ingresa la contraseña y permite añadir campos personalizados. Una vez realizada la configuración, se reinicia el dispositivo ESP32 y se conecta a la red WiFi previamente seleccionada.

Estas credenciales se guardan en la memoria flash del dispositivo, concretamente usando el **sistema de archivos SPIFFS**, que significa SPI Flash File Storage. Este sistema de archivos consiste en memoria flash muy pequeña que se encuentra en el hardware incluido en el dispositivo ESP32, y posee 1,5MB de espacio disponible. De esta manera, aunque se reinicie el dispositivo, los datos correspondientes a las credenciales del usuario permanecerán en la memoria flash para utilizarse luego del reinicio.

Del mismo modo que con las credenciales WiFi, se hizo uso de WiFiManager para gestionar las **credenciales MQTT**, que son las que permiten al dispositivo enviar las métricas al broker MQTT, validando que se trata de un dispositivo autorizado y evitando así recibir datos no deseados en los tópicos. Las credenciales MQTT se añadieron gracias a los campos personalizados (o custom) que se indican en la pantalla de configuración desde el teléfono celular. Estas credenciales también se almacenaron en la memoria flash SPIFFS.

5. Conclusiones

5.1. Evaluar los Resultados

5.1.1. Verificación de los Requerimientos

Luego de finalizar la implementación del último prototipo, se evaluaron los requerimientos propuestos.

- **RF1:** Los sensores miden correctamente la temperatura, humedad del suelo, humedad del aire y luminosidad ambiental. Las métricas se pueden visualizar en el tablero online [34].
- **RF2:** El dispositivo se conecta correctamente a internet a través de WiFi gracias a la antena integrada que posee la placa ESP32.
- **RF3:** El servidor Raspberry Pi 4 procesa y muestra las métricas de los sensores en el tablero online mencionado anteriormente.
- **RF4:** El dispositivo se actualiza automáticamente al detectar una nueva versión disponible en línea. La instalación demora aproximadamente 1 minuto, luego se reinicia y opera con el nuevo firmware actualizado.
- **RF5:** Se incluyó al flujo de trabajo el análisis de código automático ante cada merge a la rama DEV con la herramienta CodeQL. Además se incluye el analizador Cppcheck como paso previo a la compilación de cada nueva versión. Si falla alguno de estos análisis no se permite continuar con el despliegue del software.
- **RF6:** Una vez que se tiene una nueva versión del firmware del dispositivo, se debe crear un Pull Request, es decir una solicitud para mergear la rama QA a Main. Esto acciona el flujo de trabajo desencadenando los procesos de compilación, pruebas automatizadas y análisis de código. Una vez superados exitosamente, el binario resultante se coloca en la carpeta correspondiente al servicio web Nginx y se actualiza el archivo JSON que indica cual es el número de la última versión, para que las placas reconozcan y se actualicen de forma automática.
- **RF7:** El flujo de trabajo incluye un paso de pruebas automatizadas en un dispositivo real ubicado en un ambiente de pruebas.
- **RNF1:** Se realizó una medición del consumo de intensidad de corriente del dispositivo. El mismo oscila entre 70 a 110 mA dependiendo la tarea que realiza.

Puede decirse entonces que el consumo es de 110mA, cumpliendo el requisito de eficiencia energética. Este consumo puede reducirse mucho más, tomando muestras cada más tiempo e implementando el mecanismo de deep sleep.

- **RNF2:** El tamaño del dispositivo es de aproximadamente 20cm de largo por 10cm de ancho por 5cm de alto, por lo tanto cumple con el requisito de mantener un volumen reducido. El peso del dispositivo en su prototipo final es de unos 170 gramos considerando su placa de pruebas. Este peso puede disminuirse si se reemplaza la placa por una placa electrónica universal o un PCB específico.
- **RNF3:** El servidor es fácilmente replicable gracias a sus archivos Docker Compose, que permiten instanciar los servicios necesarios con un solo comando.
- **RNF4:** El dispositivo fue probado en 3 redes WiFi hogareñas distintas, en la casa de Francisco, en el departamento de Federico y en la casa sus padres. De esta forma se comprueba que el funcionamiento del dispositivo no depende de la red en la que se encuentran y que pueden funcionar en simultáneo.
- **RNF5:** Gracias a la tecnología de los contenedores de Docker es posible correr los servicios en cualquier distribución de Linux. Se probaron en las siguientes distribuciones: Ubuntu Server 22.04, ZorinOS 16.2 y LinuxMint 20.3. Funcionando en todas ellas con un desempeño similar.

5.1.2. Consideraciones Adicionales

Se puede concluir que el prototipo final cumple con el objetivo de mejorar los procesos de desarrollo de proyectos IoT. Si bien en este caso se basó en una aplicación de monitoreo de condiciones aptas para cultivo, las prácticas aplicadas y la disposición de las herramientas integradas se pueden utilizar para proyectos de distinta naturaleza.

Se logró detectar y corregir vulnerabilidades en etapas tempranas del desarrollo y las herramientas que verifican la seguridad se ejecutan en cada integración de nuevas características, lo que permite continuar un desarrollo seguro y con buenas prácticas.

Además, la automatización conseguida en el proceso de compilación, pruebas y despliegue de cada versión, permitió no sólo una reducción en los tiempos de futuros desarrollos sino también la posibilidad de tener un pipeline repetible y escalable a otras necesidades. Si alguna etapa llegase a fallar, se puede revisar detenidamente gracias a la división en pasos (o steps) del pipeline.

Este prototipo podría escalarse sin inconvenientes a numerosos clientes, con la limitación de que la potencia computacional del servidor sea suficiente para atender

las peticiones de todos los dispositivos clientes. A su vez, los servicios de monitoreo y control del servidor permiten descubrir fallos rápidamente y actuar en consecuencia.

5.2. Dificultades Encontradas

Durante el proceso de desarrollo, se presentaron las siguientes dificultades:

- Si bien la cultura DevSecOps está muy difundida y en crecimiento constante, no es muy popular aplicar las prácticas DevSecOps al desarrollo IoT, lo que significa que muchas de las características que se buscaron implementar estaban orientadas al desarrollo de software y no a la implementación en dispositivos físicos con seguridad aplicada.
- Una de las características que resultó un desafío fue la actualización por WiFi, ya que las librerías populares ofrecen la posibilidad de hacerlo de forma manual mediante un navegador web, pero hacerlo de forma automatizada sin la intervención del usuario ni del desarrollador supuso un importante desafío.
- Llevó tiempo planificar la implementación de las herramientas de seguridad en etapas tempranas, ya que por lo general al trabajar con dispositivos físicos no se suele pensar en la seguridad hasta que aparecen los inconvenientes.

5.3. Oportunidades de Mejora

Una vez finalizado el proyecto, el equipo de trabajo debatió y propuso oportunidades para continuar el desarrollo, implementando mejoras en el marco de las prácticas DevSecOps. En concreto, las mejoras considerables pueden ser:

- Orquestar los contenedores con **Kubernetes**: esta tecnología permite ejecutar los servicios en un cluster en lugar de en un único servidor, de una manera escalable. El gestor de paquetes **Helm** puede ser una herramienta útil para administrar, instalar y actualizar aplicaciones en Kubernetes.
- Considerar una alternativa para alojar los servicios en la nube, por ejemplo en **Amazon Web Services (AWS)** o **Google Cloud Platform (GCP)**. Esto permite tener alta disponibilidad y alta escalabilidad, pagando sólo por lo que se usa y no se está condicionado a usar un servidor que al volverse obsoleto se debe cambiar.
- Hay que tener en cuenta que al usar servidores en la nube ya no se puede acceder a sus puertos para cargar los tests en una placa de pruebas. Con lo cual puede

plantearse un **entorno de pruebas en emuladores como QEMU** o similares.

5.4. Agradecimientos

Expresamos nuestro agradecimiento a los docentes que nos han brindado su apoyo para realizar este trabajo, entre ellos a Ing. Miguel Ángel Solinas, Ing. Javier Jorge y al Ing. Luis Orlando Ventre, quienes con toda predisposición nos atendieron en las numerosas oportunidades que los consultamos.

A nuestros padres y abuelos por permitirnos estudiar la carrera sin preocupaciones económicas y dedicarnos completamente al estudio y aprendizaje.

A nuestros compañeros de carrera, que promovieron siempre con actitud positiva el trabajo en equipo, y siempre ayudaron ante las necesidades, tanto durante el cursado de las materias como en este proyecto integrador.

6. Bibliografía

- [1] Alldatasheet. Datasheet sensor dht11, June 2023. URL <https://pdf1.alldatasheet.com/datasheet-pdf/view/1440068/ETC/DHT11.html>.
- [2] Alldatasheet. Datasheet sensor dht22, June 2023. URL <https://pdf1.alldatasheet.com/datasheet-pdf/view/1132459/ETC2/DHT22.html>.
- [3] K. Scott Allen. *What Every Web Developer Should Know About HTTP*. OdeToCode LLC, 2012. ISBN 978-1484200773. URL <https://www.amazon.com/Every-Developer-Should-OdeToCode-Programming-ebook/dp/B0076Z6VMI>.
- [4] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010. ISSN 1389-1286. doi: <https://doi.org/10.1016/j.comnet.2010.05.010>. URL <https://www.sciencedirect.com/science/article/pii/S1389128610001568>.
- [5] Luigi Atzori, Antonio Iera, and Giacomo Morabito. Understanding the internet of things: definition, potentials, and societal role of a fast evolving paradigm. *Ad Hoc Networks*, 56:122–140, 2017. ISSN 1570-8705. doi: <https://doi.org/10.1016/j.adhoc.2016.12.004>. URL <https://www.sciencedirect.com/science/article/pii/S1570870516303316>.
- [6] Len Bass, Ingo Weber, and Liming Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015.
- [7] Jan Bosch. *Continuous Software Engineering: An Introduction*, pages 3–13. Springer International Publishing, Cham, 2014. ISBN 978-3-319-11283-1. doi: 10.1007/978-3-319-11283-1_1. URL https://doi.org/10.1007/978-3-319-11283-1_1.
- [8] Thomas Cardonne. Repositorio de github runner con docker, June 2023. URL <https://github.com/tcardonne/docker-github-runner>.
- [9] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2014. ISBN 978-1484200773. URL <https://www.amazon.com/Pro-Git-Scott-Chacon/dp/1484200772>.
- [10] Lianping Chen. Continuous delivery: Huge benefits, but challenges too. *IEEE Software*, 32(2):50–54, 2015. doi: 10.1109/MS.2015.27.
- [11] Federico Coronati. Repositorio fork de espasyncwebserver, June 2023. URL <https://github.com/FeedehC/ESPAsyncWebServer>.

- [12] Federico Coronati. Repositorio fork de pubsubclient, June 2023. URL <https://github.com/FeedehC/pubsubclient>.
- [13] Cppcheck. Cppcheck, June 2023. URL <https://cppcheck.sourceforge.io/>.
- [14] Espressif. Datasheet esp32, June 2023. URL https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [15] Espressif. Datasheet esp8266, June 2023. URL https://components101.com/sites/default/files/component_datasheet/ESP8266-NodeMCU-Datasheet.pdf.
- [16] Espressif. Guía de inicio esp-idf, June 2023. URL <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/index.html>.
- [17] Espressif. Espressif, June 2023. URL <https://www.espressif.com/en/company/about-espressif>.
- [18] David Farley and Jez Humble. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN 9780321601919.
- [19] Stephan Faßbender, Maritta Heisel, and Rene Meis. Functional requirements under security pressure. In *2014 9th International Conference on Software Paradigm Trends (ICSOFT-PT)*, pages 5–16. IEEE, 2014.
- [20] Roy Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 01 2000.
- [21] Brian Fitzgerald and Klaas-Jan Stol. Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software*, 123:176–189, 2017. ISSN 0164-1212. doi: <https://doi.org/10.1016/j.jss.2015.06.063>. URL <https://www.sciencedirect.com/science/article/pii/S0164121215001430>.
- [22] Eclipse Foundation. Mosquitto, June 2023. URL <https://mosquitto.org/>.
- [23] GitHub. Codeql, June 2023. URL <https://codeql.github.com/>.
- [24] GitHub. Billing for github actions, June 2023. URL <https://docs.github.com/es/billing/managing-billing-for-github-actions/about-billing-for-github-actions>.
- [25] GitHub. Documentación oficial de github actions, June 2023. URL <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>.

- [26] GitHub. About self-hosted runners, June 2023. URL <https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners>.
- [27] GitHub. Añadir un runner self-hosted, June 2023. URL <https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/adding-self-hosted-runners>.
- [28] GitHub. Repositorio oficial del runner de github actions, June 2023. URL <https://github.com/actions/runner>.
- [29] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. ISSN 0167-739X. doi: <https://doi.org/10.1016/j.future.2013.01.010>. URL <https://www.sciencedirect.com/science/article/pii/S0167739X13000241>. Including Special sections: Cyber-enabled Distributed Computing for Ubiquitous Cloud and Network Services and Cloud Computing and Scientific Applications — Big Data, Scalable Analytics, and Beyond.
- [30] Kelsey Hightower, Brendan Burns, and Joe Beda. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media, 2017. ISBN 978-1491935675. URL <https://www.amazon.com/-/es/Kelsey-Hightower/dp/1491935677>.
- [31] Gaston C. Hillar. *MQTT Essentials - A Lightweight IoT Protocol*. Packt Publishing, 2017. ISBN 978-1787287815. URL <https://www.amazon.com/MQTT-Essentials-Lightweight-IoT-Protocol/dp/1787287815>.
- [32] Gene Kim, Jez Humble, and John Willis. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2^a edition, August 2016. ISBN 978-1942788003. URL <https://www.amazon.com/DevOps-Handbook-World-Class-Reliability-Organizations/dp/1942788002>.
- [33] Knolleary. Repositorio de la librería pubsubclient, June 2023. URL <https://github.com/knolleary/pubsubclient>.
- [34] Francisco De la Cruz. Tablero o dashboard online de powerpot, June 2023. URL <https://dashboard.power-pot.com/>.
- [35] Leonardo Leite, Carla Rocha, Fabio Kon, Dejan Milojevic, and Paulo Meirelles. A survey of devops concepts and challenges. *ACM Comput. Surv.*, 52(6), nov 2019. ISSN 0360-0300. doi: [10.1145/3359981](https://doi.org/10.1145/3359981). URL <https://doi.org/10.1145/3359981>.

- [36] Marko Leppänen, Simo Mäkinen, Max Pagels, Veli-Pekka Eloranta, Juha Itkonen, Mika V. Mäntylä, and Tomi Männistö. The highways and country roads to continuous deployment. *IEEE Software*, 32(2):64–72, 2015. doi: 10.1109/MS.2015.50.
- [37] PVS-Studio LLC. Pvs-studio, June 2023. URL <https://pvs-studio.com/en/pvs-studio/>.
- [38] Raspberry Pi Ltd. Datasheet raspberry pi pico w, June 2023. URL <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>.
- [39] me-no dev. Repositorio de la librería espasyncwebserver, June 2023. URL <https://github.com/me-no-dev/ESPAsyncWebServer>.
- [40] Naylamp Mechatronics. Sensor capacitivo de humedad de suelo v1.2, June 2023. URL <https://naylampmechatronics.com/sensores-temperatura-y-humedad/538-sensor-de-humedad-de-suelo-capacitivo-v1.html>.
- [41] Naylamp Mechatronics. Modulo sensor de luz ldr, June 2023. URL <https://naylampmechatronics.com/sensores-luz-y-sonido/135-modulo-sensor-ldr-4-pines.html>.
- [42] Minibots. Pruebas de rendimiento de microcontroladores iot, June 2023. URL <https://minibots.wordpress.com/2021/04/06/pruebas-de-rendimiento-de-raspberry-pi-pico-y-comparativa-con-plataformas-arduino/>.
- [43] PlatformIO. Platformio, June 2023. URL <https://platformio.org/>.
- [44] PlatformIO. Documentación de platformio con cppcheck, June 2023. URL <https://docs.platformio.org/en/latest/advanced/static-code-analysis/tools/cppcheck.html>.
- [45] Nigel Poulton. *Docker Deep Dive*. Independently published, 2017. ISBN 978-1521822807. URL <https://www.amazon.com/-/es/Nigel-Poulton/dp/1521822808>.
- [46] RafaGS. Arduino speedtest, February 2018. URL <https://github.com/RafaGS/Arduino-Speedtest>.
- [47] Gerald Schermann, Jürgen Cito, Philipp Leitner, Uwe Zdun, and Harald Gall. An empirical study on principles and practices of continuous delivery and deployment. *PeerJ*, 03 2016. doi: 10.7287/peerj.preprints.1889.
- [48] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5:3909–3943, 2017.

- [49] Mojtaba Shahin, Mansooreh Zahedi, Muhammad Ali Babar, and Liming Zhu. An empirical study of architecting for continuous delivery and deployment. *Empirical Softw. Engg.*, 24(3):1061–1108, jun 2019. ISSN 1382-3256. doi: 10.1007/s10664-018-9651-4. URL <https://doi.org/10.1007/s10664-018-9651-4>.
- [50] SonarSource. Sonarcloud, June 2023. URL <https://www.sonarsource.com/products/sonarcloud/>.
- [51] SonarSource. Sonarqube, June 2023. URL <https://www.sonarsource.com/products/sonarqube/>.
- [52] Daniel Ståhl, Torvald Mårtensson, and J. Bosch. Continuous practices and devops: beyond the buzz, what does it all mean? *2017 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 440–448, 2017.
- [53] Paul Swartout. *Continuous Delivery and DevOps: A Quickstart Guide*. Packt Publishing, 2012. ISBN 9781849693684.
- [54] Synopsys. Coverity, June 2023. URL <https://scan.coverity.com/>.
- [55] The Clang Team. Clangtidy, June 2023. URL <https://clang.llvm.org/extra/clang-tidy/>.
- [56] Ingo Weber, Surya Nepal, and Liming Zhu. Developing dependable and secure cloud applications. *IEEE Internet Computing*, 20(3):74–79, 2016. doi: 10.1109/MIC.2016.67.
- [57] Wikipedia. Cmake, June 2023. URL <https://es.wikipedia.org/wiki/CMake>.
- [58] Mansooreh Zahedi, Roshan Namal Rajapakse, and Muhammad Ali Babar. Mining questions asked about continuous software engineering: A case study of stack overflow. In *Proceedings of the Evaluation and Assessment in Software Engineering*, EASE '20, page 41–50, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450377317. doi: 10.1145/3383219.3383224. URL <https://doi.org/10.1145/3383219.3383224>.